

OpenRNG

Computational Trust Infrastructure



計算信任基礎設施

Version 1.0

June 2026

Bilingual Edition — English / 繁體中文

CONFIDENTIAL — FOR IP COUNSEL REVIEW

Contents 目錄

PART I — THE PROBLEM 問題

Chapter 1 — The Computational Trust Problem 計算信任問題

- 1.1 The Trust Gap in Autonomous Systems 自主系統中的信任缺口
- 1.2 Why This Is Not a Security Problem 為什麼這不是安全問題
- 1.3 Why This Is Not a Randomness Problem 為什麼這不是隨機性問題
- 1.4 Defining Computational Trust 定義計算信任
- 1.5 The Inevitability Argument 必然性論證

Chapter 2 — Why Existing Infrastructure Falls Short 為什麼現有基礎設施不足

- 2.1 Transport Security (TLS / mTLS)
- 2.2 Authentication & Authorization (OAuth, JWT, RBAC)
- 2.3 Audit Logs 稽核日誌
- 2.4 Verifiable Random Functions (VRF)
- 2.5 Trusted Execution Environments (TEE)
- 2.6 Multi-Party Computation (MPC)
- 2.7 Blockchain Consensus 區塊鏈共識
- 2.8 The Gap Map 缺口地圖

PART II — THE SOLUTION 解決方案

Chapter 3 — The OpenRNG Trust Stack 信任堆疊架構

- 3.1 Design Principles 設計原則
- 3.2 The Five-Layer Stack 五層堆疊架構
- 3.3 Layer 1: Trust Source (OpenRNG Core) 信任來源
- 3.4 Layer 2: Trust Authorization (Lockbox) 信任授權
- 3.5 Layer 3: Trust Evidence (VEO) 信任證據
- 3.6 Layer 4: Trust Coordination 信任協調
- 3.7 Layer 5: Trust Runtime 信任運行時
- 3.8 Cross-Cutting Concerns 跨層關注事項

Chapter 4 — The Trust Lifecycle 信任生命週期

- 4.1 Why a Lifecycle? 為什麼需要生命週期？
- 4.2 The Eight Stages 八個階段
- 4.3 Lifecycle Profiles 生命週期配置
- 4.4 Lifecycle Invariants 生命週期不變量
- 4.5 The Lifecycle as Category-Defining Primitive 作為類別定義原語

PART III — THE MARKET 市場

Chapter 5 — The Patent Family 專利家族

- 5.1 Family Overview 家族概覽
- 5.2 Patent #1: Trust Source Layer 信任來源層
- 5.3 Patent #2: Trust Authorization Layer 信任授權層
- 5.4 Patent #3: Trust Evidence Layer 信任證據層
- 5.5 Patent #4: Trust Coordination Layer 信任協調層
- 5.6 Patent #5: Trust Runtime 信任運行時
- 5.7 Future Patent Opportunities 未來專利機會
- 5.8 Family Coherence 家族一致性

Chapter 6 — Markets 市場

- 6.1 AI Agents (Primary) · 6.2 Enterprise AI
- 6.3 Gaming · 6.4 Robotics · 6.5 Government
- 6.6 Finance · 6.7 Healthcare · 6.8 Scientific Computing

Chapter 7 — Why This Matters 為什麼這很重要

- 7.1 The Historical Pattern 歷史模式
- 7.2 Computational Trust Is the Next Category 下一個類別
- 7.3 The Infrastructure Test 基礎設施測試
- 7.4 The Five-Year Vision 五年願景
- 7.5 What Success Looks Like 成功的樣貌

PART I

The Problem

問題

The Computational Trust Problem

計算信任問題

Why AI needs a new infrastructure layer

為什麼 AI 需要一個新的基礎設施層

1.1 — The Trust Gap in Autonomous Systems

1.1 — 自主系統中的信任缺口

Machines are making decisions with real consequences. An AI agent selects which medical records to surface for a physician. A trading algorithm decides when to execute a block order. A content moderation system determines which posts reach millions of users. A robotic warehouse system allocates tasks among hundreds of autonomous units.

These decisions share a common structure: they involve non-deterministic processes — randomness, sampling, probabilistic selection — and they produce outcomes that affect real people, real money, and real systems. Yet in every case, the process that produced the decision is invisible. The decision arrives. The outcome is delivered. But the computational path that led from input to output — including any random or non-deterministic steps — is gone. It was never recorded in a verifiable form. It cannot be independently audited. It cannot be proven fair.

This is the trust gap. Not a gap in security (the systems are well-protected), not a gap in capability (the systems perform well), but a gap in **verifiability**. No independent party can examine the computational process after the fact and confirm that it operated correctly, fairly, and without manipulation.

For decades, this gap did not matter. Computational decisions were made by deterministic software under direct human supervision. If a question arose, a human could explain the decision. But the rise of autonomous AI systems — agents that act independently, at scale, without human-in-the-loop oversight — has made this gap untenable. When no human is watching, and the machine's decisions have material consequences, the question "can we verify what happened?" becomes existential.

機器正在做出具有真實後果的決策。AI 代理選擇為醫生呈現哪些醫療記錄。交易演算法決定何時執行大宗訂單。內容審核系統決定哪些貼文觸及數百萬用戶。機器人倉庫系統在數百個自主單元之間分配任務。

這些決策有著共同的結構：它們涉及非確定性過程——隨機性、抽樣、機率選擇——並產生影響真實的人、真實的資金和真實系統的結果。然而在每種情況下，產生決策的過程都是不可見的。決策到達了。結果被交付了。但從輸入到輸出的計算路徑——包括任何隨機或非確定性步驟——已經消失。它從未以可驗證的形式被記錄。它無法被獨立稽核。它無法被證明是公平的。

這就是信任缺口。不是安全缺口（系統受到良好保護），不是能力缺口（系統表現良好），而是**可驗證性**的缺口。沒有獨立方可以在事後檢查計算過程並確認其運作正確、公平且未被操縱。

幾十年來，這個缺口無關緊要。計算決策由確定性軟體在直接人類監督下做出。如果出現問題，人類可以解釋決策。但自主 AI 系統的興起——獨立行動、大規模運作、無需人類介入監督的代理——使這個缺口變得不可持續。當沒有人類在觀察，而機器的決策具有重大後果時，「我們能否驗證發生了什麼？」這個問題變得至關重要。

1.2 — Why This Is Not a Security Problem

1.2 — 為什麼這不是安全問題

The instinct when confronting a trust gap is to reach for security tools: encryption, access control, firewalls, intrusion detection. But the Computational Trust problem is fundamentally different from a security problem.

Security prevents unauthorized access. It asks: "Can an attacker breach this system?" The tools are encryption (protecting data in transit and at rest), authentication (verifying identity), authorization (controlling access), and monitoring (detecting intrusions). Security assumes that if the system is well-protected, its outputs can be trusted.

Privacy prevents unauthorized disclosure. It asks: "Can sensitive data be exposed?" The tools are anonymization, differential privacy, data minimization, and consent management. Privacy assumes that if data is properly protected, its handling can be trusted.

Trust proves that a process occurred correctly. It asks: "Can we verify, after the fact, that this computation was performed fairly, with proper inputs, and without manipulation — even by the operator?" This question cannot be answered by encryption (the data is not being stolen), by

access control (the operator is authorized), or by monitoring (the system is not being attacked). The system is operating normally. The question is whether "normally" means "correctly and verifiably."

The difference is structural. Security protects the perimeter. Trust proves the interior. A perfectly secure system can still produce decisions that cannot be independently verified — and in a world of autonomous AI, unverifiable decisions are unacceptable regardless of how secure the system is.

面對信任缺口時的直覺反應是使用安全工具：加密、存取控制、防火牆、入侵偵測。但計算信任問題與安全問題根本不同。

安全防止未經授權的存取。它問的是：「攻擊者能否突破此系統？」工具是加密（保護傳輸中和靜態的資料）、認證（驗證身份）、授權（控制存取）和監控（偵測入侵）。安全假設如果系統受到良好保護，其輸出就可以被信任。

隱私防止未經授權的洩露。它問的是：「敏感資料能否被暴露？」工具是匿名化、差分隱私、資料最小化和同意管理。隱私假設如果資料受到適當保護，其處理就可以被信任。

信任證明過程正確發生。它問的是：「我們能否在事後驗證此計算是公平執行的、使用了適當的輸入、且未被操縱——即使是被操作者自己？」這個問題無法由加密回答（資料沒有被竊取）、無法由存取控制回答（操作者是被授權的）、也無法由監控回答（系統沒有被攻擊）。系統正常運作。問題是「正常」是否意味著「正確且可驗證」。

差異是結構性的。安全保護邊界。信任證明內部。一個完全安全的系統仍然可以產生無法獨立驗證的決策——而在自主 AI 的世界中，不可驗證的決策是不可接受的，無論系統有多安全。

1.3 — Why This Is Not a Randomness Problem

1.3 — 為什麼這不是隨機性問題

Random number generation is a solved problem. Every programming language has a built-in RNG. Every operating system provides cryptographic randomness through `/dev/urandom` or equivalent. Every cloud provider offers randomness as a service. The world does not need another random number generator.

The problem is not generation. The problem is **trust**.

When a traditional RNG produces a number, the number arrives and the process disappears. The consumer receives a value but cannot answer three fundamental questions: Where did this randomness come from? Was it generated fairly? Can anyone verify this after the fact? These questions are unanswerable because traditional RNG was never designed to answer them. The random number is the product; the process is discarded.

Randomness is merely the first domain where the Computational Trust gap becomes critical — because randomness is inherently unobservable. You cannot "look at" a random number and determine whether it was truly random, whether it was manipulated, or whether it came from a single weak source or multiple independent strong sources. This makes randomness the hardest case for trust, and therefore the right place to build the trust infrastructure that will eventually extend to all non-deterministic computation.

The vision is not "better randomness." The vision is **Computational Trust as infrastructure** — starting with entropy (the hardest case) and extending to decisions, computations, and processes of all kinds.

隨機數生成是一個已解決的問題。每種程式語言都有內建的 RNG。每個作業系統透過 `/dev/urandom` 或等效方式提供密碼學隨機性。每個雲端供應商都提供隨機性即服務。世界不需要另一個隨機數生成器。

問題不是生成。問題是**信任**。

當傳統 RNG 產生一個數字時，數字到達了但過程消失了。消費者收到一個值但無法回答三個基本問題：這個隨機性從哪裡來？它是公平生成的嗎？事後有人能驗證嗎？這些問題無法回答，因為傳統 RNG 從未被設計來回答它們。隨機數是產品；過程被丟棄了。

隨機性只是計算信任缺口變得關鍵的第一個領域——因為隨機性本質上是不可觀察的。你無法「看著」一個隨機數並判斷它是否真正隨機、是否被操縱、或者它來自單一弱來源還是多個獨立強來源。這使得隨機性成為信任最困難的案例，因此也是建構信任基礎設施的正確起點——這個基礎設施最終將擴展到所有非確定性計算。

願景不是「更好的隨機性」。願景是**計算信任作為基礎設施**——從熵（最困難的案例）開始，擴展到所有類型的決策、計算和過程。

1.4 — Defining Computational Trust

1.4 — 定義計算信任

Computational Trust is the ability to independently verify that a computational process — its inputs, execution, and outputs — occurred correctly, fairly, and without manipulation, without relying on the operator's assertion.

This definition has three critical components:

Independently — verification does not require cooperation from the operator. A third party with access to the trust evidence can verify everything without the operator's involvement, permission, or even awareness. This is what distinguishes Computational Trust from audit logging: an audit log is written by the operator; a trust object is verified by anyone.

The entire process — not just the output, but the inputs (where did the data come from?), the execution (was the correct algorithm applied?), and the outputs (was the result delivered without modification?). Partial trust is insufficient. A system that proves its output is correct but cannot prove its inputs were fair provides a false sense of assurance.

Without relying on the operator's assertion — the operator may be honest, compromised, or adversarial. Computational Trust works in all three cases. It is not a trust framework (which assumes good faith); it is a verification framework (which assumes nothing).

Computational Trust rests on three pillars:

Provenance: Where did the inputs come from? Every input to a trusted computation must be traceable to its source, with cryptographic evidence linking each step. Provenance answers the question "was this built on legitimate foundations?"

Integrity: Was the process tampered with? Cryptographic hashes and signatures ensure that any modification to inputs, intermediate values, or outputs is detectable. Integrity answers the question "was this altered after creation?"

Auditability: Can a third party reconstruct and verify what happened? Every step in the process produces a verifiable artifact that can be examined independently, in any order, at any time in the future. Auditability answers the question "can we prove this to someone who wasn't there?"

計算信任是獨立驗證計算過程——其輸入、執行和輸出——正確、公平且未被操縱地發生的能力，無需依賴操作者的聲明。

此定義有三個關鍵組成部分：

獨立地——驗證不需要操作者的配合。擁有信任證據的第三方可以在不需要操作者參與、許可甚至知曉的情況下驗證一切。這就是計算信任與稽核日誌的區別：稽核日誌由操作者撰寫；信任物件由任何人驗證。

整個過程——不僅是輸出，還有輸入（資料從哪裡來？）、執行（是否使用了正確的演算法？）和輸出（結果是否未經修改地交付？）。部分信任是不夠的。一個能證明輸出正確但無法證明輸入公平的系統提供的是虛假的保證。

無需依賴操作者的聲明——操作者可能是誠實的、被入侵的或是對抗性的。計算信任在所有三種情況下都有效。它不是信任框架（假設善意）；它是驗證框架（不做任何假設）。

計算信任建立在三個支柱之上：

出處 (Provenance)：輸入從哪裡來？受信任計算的每個輸入都必須可追溯到其來源，每個步驟都有密碼學證據連結。出處回答「這是建立在合法基礎上的嗎？」

完整性 (Integrity)：過程是否被篡改？密碼學雜湊和簽名確保對輸入、中間值或輸出的任何修改都可被偵測。完整性回答「這在建立後是否被更改？」

可稽核性 (Auditability)：第三方能否重建並驗證所發生的事情？過程中的每一步都產生可驗證的成品，可以獨立地、以任何順序、在未來任何時間進行檢查。可稽核性回答「我們能否向不在場的人證明這件事？」

1.5 — The Inevitability Argument

1.5 — 必然性論證

Three converging forces make Computational Trust Infrastructure inevitable:

Force 1: AI Agents Are Gaining Authority. The trajectory is clear: AI agents are moving from advisory roles (suggesting actions for humans to approve) to autonomous roles (taking actions independently). OpenAI's agents can execute multi-step workflows. Anthropic's agents can use

computers. Google's agents can manage enterprise processes. As these agents gain authority over financial transactions, medical decisions, legal processes, and physical systems, their decisions will require the same level of verification that human decisions require today. There is no infrastructure for this.

Force 2: Regulatory Frameworks Are Converging. The EU AI Act (effective 2026) requires transparency and auditability for high-risk AI systems. The NIST AI Risk Management Framework demands governance and accountability. ISO 42001 establishes management system requirements for AI. The SEC is examining AI-driven trading decisions. Healthcare regulators are scrutinizing AI triage systems. Every major regulatory body is independently arriving at the same requirement: AI decisions must be verifiable. But none of them specifies *how* — because the infrastructure to do so does not yet exist.

Force 3: No Existing Infrastructure Provides This. As Chapter 2 will detail, the current technology landscape — TLS, OAuth, TEE, VRF, MPC, blockchain consensus, audit logs — provides individual components of trust but does not connect them into a verifiable lifecycle. Each technology solves its piece of the puzzle. None solves the puzzle. The gap is not that existing technologies are insufficient; it is that no one has built the layer that connects them.

The convergence is clear: autonomous systems are proliferating, regulators are demanding verifiability, and no infrastructure exists to provide it. This is the classic precondition for a new infrastructure category — the same precondition that preceded TLS (the web needed encrypted communication), OAuth (applications needed delegated authorization), and containerization (applications needed portable deployment). Computational Trust is the next category. The question is not whether it will be built, but by whom.

三股匯聚的力量使計算信任基礎設施不可避免：

力量一：AI 代理正在獲得權限。軌跡是明確的：AI 代理正從諮詢角色（為人類建議需批准的行動）轉向自主角色（獨立採取行動）。OpenAI 的代理可以執行多步驟工作流程。Anthropic 的代理可以使用電腦。Google 的代理可以管理企業流程。隨著這些代理對金融交易、醫療決策、法律程序和實體系統獲得權限，它們的決策將需要與今天人類決策相同水準的驗證。目前沒有這方面的基礎設施。

力量二：監管框架正在趨同。EU AI Act（2026 年生效）要求高風險 AI 系統的透明度和可稽核性。NIST AI 風險管理框架要求治理和問責。ISO 42001 建立 AI 管理系統要求。SEC 正在審查 AI 驅動的交易決策。醫療保健監管機構正在審視 AI 分診系統。每個主要監管機構都獨立地得出相同的要求：AI 決策必須可驗證。但沒有一個指定*如何*做到——因為做到這一點的基礎設施尚不存在。

力量三：沒有現有基礎設施提供這一點。如第二章將詳述的，當前的技術版圖——TLS、OAuth、TEE、VRF、MPC、區塊鏈共識、稽核日誌——提供信任的個別組件，但沒有將它們連接成可驗證的生命週期。每種技術解決了拼圖的一塊。沒有一個解決了整個拼圖。缺口不是現有技術不足，而是沒有人建構了連接它們的那一層。

趨同是明確的：自主系統正在擴散、監管機構正在要求可驗證性、而沒有基礎設施來提供它。這是新基礎設施類別的典型前提條件——與 TLS（網路需要加密通訊）、OAuth（應用程式需要委託授權）和容器化（應用程式需要可攜帶部署）之前的前提條件相同。計算信任是下一個類別。問題不是它是否會被建構，而是由誰來建構。

Why Existing Infrastructure Falls Short 為什麼現有基礎設施不足

A systematic analysis of the current trust landscape

當前信任格局的系統性分析

The technologies examined in this chapter are not failing. They are excellent solutions to the problems they were designed to solve. The point of this analysis is not to diminish them but to demonstrate that Computational Trust is a *distinct problem* that none of them were designed to address. They are complementary layers in a complete trust architecture — but the trust layer itself is missing.

本章檢視的技術並沒有失敗。它們是為其設計目的而設的優秀解決方案。本分析的目的不是要貶低它們，而是要證明計算信任是一個獨特的問題，它們都不是為解決這個問題而設計的。在完整的信任架構中，它們是互補的層——但信任層本身是缺失的。

2.1 — Transport Security (TLS / mTLS)

2.1 — 傳輸安全 (TLS / mTLS)

What it solves: TLS (Transport Layer Security) provides encrypted, authenticated communication between two parties. It ensures that data in transit cannot be intercepted, read, or modified by third parties. mTLS extends this with mutual authentication — both parties verify each other's identity.

What it does not solve: TLS proves that the pipe is secure. It says nothing about the water flowing through the pipe. A perfectly encrypted channel can transmit a manipulated random number, a biased AI decision, or a falsified audit record. TLS guarantees that no one *outside* the communication tampered with the data. It provides no guarantee about what the *sender* did before transmitting.

The gap: TLS is perimeter security for communication. Computational Trust requires process verification for computation. They operate at different layers and solve different problems.

它解決什麼：TLS（傳輸層安全）提供兩方之間的加密、認證通訊。它確保傳輸中的資料不會被第三方攔截、讀取或修改。mTLS 透過相互認證擴展——雙方都驗證對方的身份。

它不解決什麼：TLS 證明管道是安全的。它對流過管道的水不做任何聲明。一個完全加密的通道可以傳輸被操縱的隨機數、有偏見的 AI 決策或偽造的稽核記錄。TLS 保證通訊外部沒有人篡改資料。它不保證發送者在傳輸前做了什麼。

缺口：TLS 是通訊的邊界安全。計算信任需要的是計算的過程驗證。它們運作在不同的層級，解決不同的問題。

2.2 — Authentication & Authorization (OAuth, JWT, RBAC)

2.2 — 認證與授權（OAuth、JWT、RBAC）

What they solve: OAuth provides delegated authorization — a user grants an application limited access to their resources. JWT (JSON Web Tokens) provides portable, signed identity claims. RBAC (Role-Based Access Control) restricts actions based on assigned roles. Together, they answer: "Who is this user, and what are they allowed to do?"

What they do not solve: Whether what the authorized user *did* was correct, fair, or verifiable. An AI agent may be fully authorized via OAuth to make trading decisions. RBAC may correctly grant it access to the trading API. JWT may correctly assert its identity. None of these mechanisms verify that the trading decision itself was made using fair, unmanipulated inputs. Authorization asks "may you act?" Trust asks "did you act correctly?"

The gap: Authorization is a gate — it controls entry. Computational Trust is a proof system — it verifies the process after entry. A fully authorized actor can still produce unverifiable decisions.

它們解決什麼：OAuth 提供委託授權——用戶授予應用程式有限的資源存取權。JWT (JSON Web Tokens) 提供可攜帶、已簽名的身份聲明。RBAC (基於角色的存取控制) 根據分配的角色限制操作。它們共同回答：「這個用戶是誰，他們被允許做什麼？」

它們不解決什麼：被授權的用戶所做的是否正確、公平或可驗證。AI 代理可能完全透過 OAuth 被授權做出交易決策。RBAC 可能正確地授予它存取交易 API 的權限。JWT 可能正確地聲明它的身份。這些機制都不驗證交易決策本身是否使用了公平、未被操縱的輸入。授權問的是「你可以行動嗎？」信任問的是「你的行動正確嗎？」

缺口：授權是閘門——它控制進入。計算信任是證明系統——它在進入後驗證過程。一個完全被授權的行為者仍然可以產生不可驗證的決策。

2.3 — Audit Logs

2.3 — 稽核日誌

What they solve: Audit logs record that events occurred — who did what, when, and to which resource. They are the standard mechanism for after-the-fact examination in enterprise systems, cloud platforms, and regulated industries.

What they do not solve: Audit logs are written by the same system they are supposed to audit. This creates a circular trust dependency: to trust the log, you must trust the system that wrote the log. But if you already trust the system, you don't need the log. And if you don't trust the system, you can't trust the log either. A compromised or adversarial operator can modify audit logs, selectively omit entries, or generate fabricated records. Append-only logs and log aggregation services mitigate but do not eliminate this problem — the aggregation service itself becomes another trusted party.

The gap: Audit logs are operator-written records. Trust evidence is independently verifiable proof. The difference is the direction of trust: logs require you to trust the writer; trust objects require you to trust only mathematics.

它解決什麼：稽核日誌記錄事件的發生——誰做了什麼、何時、對哪個資源。它們是企業系統、雲端平台和受監管行業中事後檢查的標準機制。

它不解決什麼：稽核日誌由它們應該稽核的同一系統撰寫。這造成了循環信任依賴：要信任日誌，你必須信任撰寫日誌的系統。但如果你已經信任系統，你就不需要日誌。如果你不信任系統，你也不能信任日誌。被入侵或對抗性的操作者可以修改稽核日誌、選擇性地遺漏條目，或生成偽造的記錄。僅可追加日誌和日誌聚合服務緩解但不能消除此問題——聚合服務本身成為另一個需要信任的方。

缺口：稽核日誌是操作者撰寫的記錄。信任證據是可獨立驗證的證明。差異在於信任的方向：日誌要求你信任撰寫者；信任物件只要求你信任數學。

2.4 — Verifiable Random Functions (VRF)

2.4 — 可驗證隨機函數 (VRF)

What they solve: A VRF (such as Chainlink VRF) produces a random output along with a proof that the output was correctly derived from a specific input and a specific secret key. This proof can be verified on-chain, ensuring that the randomness provider cannot manipulate the result after seeing the input.

What they do not solve: VRFs prove *derivation*, not *trust*. Specifically: (1) VRFs typically use a single entropy source — they do not aggregate multiple independent sources or measure source diversity. (2) VRFs do not score entropy quality — there is no confidence metric indicating how trustworthy the entropy is. (3) VRFs require on-chain callback for verification — they cannot be verified offline. (4) VRFs do not produce self-describing objects — the proof is tied to a specific contract and chain, not portable across systems. (5) VRFs do not carry provenance metadata — the consumer knows the output is correctly derived but does not know the quality or independence of the underlying entropy.

The gap: VRF is a cryptographic primitive. Computational Trust is an infrastructure layer. VRF proves that a function was computed correctly; it does not prove that the inputs, quality, and lifecycle were trustworthy. OpenRNG can use VRF as one of its source verification mechanisms — the layers are complementary, not competing.

它解決什麼：VRF（如 Chainlink VRF）產生一個隨機輸出以及一個證明，證明該輸出是從特定輸入和特定秘密金鑰正確導出的。此證明可以在鏈上驗證，確保隨機性供應商在看到輸入後無法操縱結果。

它不解決什麼：VRF 證明的是導出，不是信任。具體而言：(1) VRF 通常使用單一熵源——它不聚合多個獨立來源或衡量來源多樣性。(2) VRF 不對熵品質評分——沒有信心指標表明熵有多值得信賴。(3) VRF 需要鏈上回呼進行驗證——無法離線驗證。(4) VRF 不產生自描述物件——證明綁定到特定合約和鏈，無法跨系統攜帶。(5) VRF 不攜帶出處元數據——消費者知道輸出被正確導出，但不知道底層熵的品質或獨立性。

缺口：VRF 是密碼學原語。計算信任是基礎設施層。VRF 證明函數被正確計算；它不證明輸入、品質和生命週期是可信賴的。OpenRNG 可以使用 VRF 作為其來源驗證機制之一——這些層是互補的，不是競爭的。

2.5 — Trusted Execution Environments (TEE)

2.5 — 可信執行環境（TEE）

What they solve: TEEs (Intel SGX, ARM TrustZone, AWS Nitro Enclaves) provide hardware-isolated execution environments that even the host operating system and cloud provider cannot inspect or tamper with. Code running inside a TEE produces an attestation proving it ran in a protected environment.

What they do not solve: TEEs prove execution *isolation*, not process *correctness*. A TEE guarantees that the code inside was not tampered with during execution. It does not guarantee that the code is correct, that the inputs were fair, or that the outputs are trustworthy. A TEE can faithfully execute a biased algorithm on manipulated inputs and produce a valid attestation. The attestation proves the environment was secure; it says nothing about the computation's fairness or the quality of its non-deterministic components.

The gap: TEE protects the execution environment. Computational Trust verifies the computational process. A TEE can host a Trust Stack implementation — providing hardware-level assurance that the trust logic itself was not tampered with — but the TEE alone does not provide computational trust.

它解決什麼：TEE (Intel SGX、ARM TrustZone、AWS Nitro Enclaves) 提供硬體隔離的執行環境，即使主機作業系統和雲端供應商也無法檢查或篡改。在 TEE 內運行的程式碼產生證明其在受保護環境中運行的證明。

它不解決什麼：TEE 證明的是執行 *隔離*，不是過程 *正確性*。TEE 保證內部的程式碼在執行期間未被篡改。它不保證程式碼是正確的、輸入是公平的，或者輸出是可信賴的。TEE 可以在被操縱的輸入上忠實地執行有偏見的演算法，並產生有效的證明。證明證明的是環境是安全的；它對計算的公平性或其非確定性組件的品質不做任何說明。

缺口：TEE 保護執行環境。計算信任驗證計算過程。TEE 可以託管信任堆疊實作——提供硬體層級的保證，確保信任邏輯本身未被篡改——但 TEE 本身不提供計算信任。

2.6 — Multi-Party Computation (MPC)

2.6 — 多方計算 (MPC)

What it solves: MPC enables multiple parties to jointly compute a function over their private inputs without revealing those inputs to each other. It is used in privacy-preserving analytics, secure auctions, and threshold cryptography.

What it does not solve: MPC proves that a computation was performed correctly across parties without revealing inputs. However, MPC does not produce portable, independently verifiable trust evidence. The proof of correct computation is internal to the MPC protocol — a third party who was not a participant cannot independently verify the result without re-running the protocol. MPC also does not address entropy quality, source provenance, or trust lifecycle management.

The gap: MPC is a computation protocol for privacy. Computational Trust is a verification infrastructure for auditability. MPC can be a component within the Trust Stack — for example, multi-party entropy generation could use MPC to ensure no single party controls the randomness — but MPC alone does not provide computational trust.

它解決什麼：MPC 使多方能夠在各自的私有輸入上共同計算一個函數，而不向彼此揭露這些輸入。它用於隱私保護分析、安全拍賣和門限密碼學。

它不解決什麼：MPC 證明計算在各方之間正確執行，而不揭露輸入。然而，MPC 不產生可攜帶、可獨立驗證的信任證據。正確計算的證明是 MPC 協議內部的——不是參與者的第三方無法在不重新運行協議的情況下獨立驗證結果。MPC 也不涉及熵品質、來源出處或信任生命週期管理。

缺口：MPC 是用於隱私的計算協議。計算信任是用於可稽核性的驗證基礎設施。MPC 可以是信任堆疊中的一個組件——例如，多方熵生成可以使用 MPC 來確保沒有單一方控制隨機性——但 MPC 本身不提供計算信任。

2.7 — Blockchain Consensus

2.7 — 區塊鏈共識

What it solves: Blockchain consensus mechanisms (Proof of Work, Proof of Stake, BFT variants) enable untrusted parties to agree on a shared state without a central authority. They provide immutable, append-only ledgers that no single party can alter retroactively.

What it does not solve: Blockchain consensus proves that on-chain state transitions are valid. It does not prove that off-chain computations — the vast majority of real-world computing — were performed correctly. An AI agent making decisions off-chain can record a hash on-chain, but the hash only proves that something existed at a point in time. It does not prove that the computation was fair, that the inputs were unmanipulated, or that the entropy was high-quality. Blockchain provides temporal anchoring and immutability — valuable components of trust — but not the full trust lifecycle.

The gap: Blockchain is an anchoring mechanism. Computational Trust uses blockchain as one component (Layer 3 anchoring) but provides the full lifecycle — from source generation through evidence construction through coordination through governance — that blockchain alone cannot.

它解決什麼：區塊鏈共識機制（工作量證明、權益證明、BFT 變體）使不受信任的各方能夠在沒有中央權威的情況下就共享狀態達成一致。它們提供不可變的、僅可追加的帳本，任何單一方都無法追溯更改。

它不解決什麼：區塊鏈共識證明鏈上狀態轉換是有效的。它不證明鏈下計算——現實世界計算的絕大多數——是正確執行的。AI 代理在鏈下做出決策可以在鏈上記錄雜湊，但雜湊只證明某物在某個時間點存在。它不證明計算是公平的、輸入未被操縱，或者熵是高品質的。區塊鏈提供時間錨定和不可變性——信任的有價值組件——但不是完整的信任生命週期。

缺口：區塊鏈是錨定機制。計算信任使用區塊鏈作為一個組件（第三層錨定），但提供了區塊鏈單獨無法提供的完整生命週期——從來源生成到證據建構到協調到治理。

2.8 — The Gap Map

2.8 — 缺口地圖

The following table summarizes the trust capabilities of each technology examined. Six properties define Computational Trust. No existing technology provides all six.

下表總結了所檢視的每種技術的信任能力。六個屬性定義計算信任。沒有現有技術提供全部六項。

Technology 技術	Provenance 出處	Integrity 完整性	Auditability 可稽核性	Quality Scoring 品質評分	Offline Verification 離線驗證	Standard Object 標準 物件
TLS / mTLS	—	✓	—	—	—	—
OAuth / JWT / RBAC	Partial 部分	✓	Partial 部分	—	✓ (JWT)	✓ (JWT)
Audit Logs	Partial 部分	—	Circular 循 環	—	—	—
VRF	—	✓	On-chain 鏈上	—	—	—
TEE	—	✓	Partial 部分	—	—	—
MPC	—	✓	—	—	—	—
Blockchain	—	✓	On-chain 鏈上	—	—	—
Computational Trust (VEO)	✓	✓	✓	✓	✓	✓

Conclusion: Each existing technology solves a real and important problem. None was designed to solve Computational Trust. They are complementary layers — TLS secures the transport, OAuth controls access, TEE protects the execution environment, blockchain provides anchoring. What is missing is the layer that ties them together into a verifiable trust lifecycle: the layer that generates trust material from multiple sources, measures its quality, packages it as a portable proof, propagates it across agents, enforces governance, and enables independent audit.

That missing layer is Computational Trust Infrastructure. OpenRNG is its first implementation.

結論：每種現有技術都解決了一個真實而重要的問題。沒有一種是為解決計算信任而設計的。它們是互補的層——TLS 保護傳輸、OAuth 控制存取、TEE 保護執行環境、區塊鏈提供錨定。缺失的是將它們連結成可驗證信任生命週期的那一層：從多個來源生成信任材料、衡量其品質、將其封裝為可攜帶的證明、在代理之間傳播、執行治理，並實現獨立稽核的那一層。

那個缺失的層就是計算信任基礎設施。OpenRNG 是其第一個實作。

PART II

The Solution

解決方案

The OpenRNG Trust Stack

OpenRNG 信任堆疊架構

Architecture of a Computational Trust Infrastructure

計算信任基礎設施的架構設計

3.1 — Design Principles

3.1 — 設計原則

Before describing the architecture, we must establish the principles that constrain it. These principles are not aspirational — they are engineering requirements. Any architectural decision that violates a principle is a defect.

Principle 1: Trust Must Be Portable. A trust object must be verifiable anywhere — across runtimes, across organizations, across jurisdictions. A VEO generated on AWS must be verifiable on Azure, on-premises, or on a developer's laptop. Trust cannot be locked to a platform, a chain, or a provider. This is the portability property that makes JWT useful: a JWT issued by Auth0 can be verified by any service with the public key. Trust objects must have the same property.

Principle 2: Trust Must Be Independently Verifiable. Verification must never require contacting the issuer. No callbacks, no phone-home, no "check with our API." A trust object carries its own proof. Given the object and the provider's public key, any party can verify it — offline, air-gapped, years later. This is what distinguishes infrastructure from a service: infrastructure works even when the provider is unreachable.

Principle 3: Trust Must Be Composable. A trust object from one computation can be an input to another computation, creating trust chains. An AI agent that receives a VEO can use it to make a decision, produce a new VEO for that decision, and pass it to the next agent. Each link in the chain is independently verifiable. Composability is what makes trust scale — without it, every interaction requires a fresh root of trust.

Principle 4: Trust Must Be Measurable. Trust is not binary. A trust object backed by three independent entropy sources and blockchain anchoring is more trustworthy than one backed by a single PRNG. The Entropy Confidence Score (ECS) quantifies this difference across six

dimensions, producing a score from 0 to 1000 with defined grade boundaries. Measurability enables policy: "this application requires AAA-grade trust" becomes an enforceable constraint.

Principle 5: Trust Must Be Standard. A proprietary trust format is not infrastructure — it is a product. For Computational Trust to become a category, the object format, scoring system, signing protocol, and verification process must be open, documented, and implementable by anyone. The VEO-1 specification, ECS v1, and the canonical signing protocol are designed as open standards from the outset.

在描述架構之前，我們必須建立約束架構的原則。這些原則不是願景——而是工程需求。任何違反原則的架構決策都是缺陷。

原則一：信任必須可攜帶。信任物件必須在任何地方都可驗證——跨執行環境、跨組織、跨司法管轄區。在 AWS 上生成的 VEO 必須能在 Azure、本地環境或開發者的筆記型電腦上驗證。信任不能被鎖定在特定平台、鏈或供應商。這就是讓 JWT 有用的可攜帶性：Auth0 發行的 JWT 可以被任何擁有公鑰的服務驗證。信任物件必須具有相同的性質。

原則二：信任必須可獨立驗證。驗證永遠不需要聯繫發行者。沒有回呼、沒有回報、沒有「請向我們的 API 確認」。信任物件自帶證明。給定物件和供應商的公鑰，任何一方都可以驗證——離線、氣隙隔離、甚至多年之後。這就是基礎設施與服務的區別：基礎設施在供應商無法連線時仍然運作。

原則三：信任必須可組合。一個計算的信任物件可以成為另一個計算的輸入，形成信任鏈。收到 VEO 的 AI 代理可以用它做出決策，為該決策產生新的 VEO，並傳遞給下一個代理。鏈中的每個環節都可獨立驗證。可組合性讓信任得以擴展——沒有它，每次互動都需要全新的信任根。

原則四：信任必須可量測。信任不是二元的。由三個獨立熵源和區塊鏈錨定支持的信任物件，比僅由單一 PRNG 支持的信任物件更值得信賴。熵信心分數（ECS）跨六個維度量化這個差異，產生 0 到 1000 的分數，並定義分級邊界。可量測性使策略成為可能：「此應用程式需要 AAA 級信任」成為可執行的約束。

原則五：信任必須是標準。專有的信任格式不是基礎設施——而是產品。要讓計算信任成為一個類別，物件格式、評分系統、簽名協議和驗證流程必須是開放的、有文件記錄的、任何人都可以實作的。VEO-1 規範、ECS v1 和規範簽名協議從一開始就被設計為開放標準。

3.2 — The Five-Layer Stack

3.2 — 五層堆疊架構

The OpenRNG Trust Stack is a five-layer architecture. Each layer has a single responsibility, defined interfaces, and clean boundaries with adjacent layers. The layers are numbered bottom-up: Layer 1 is the foundation, Layer 5 is the governance surface.

OpenRNG 信任堆疊是一個五層架構。每一層都有單一職責、定義明確的介面，以及與相鄰層的清晰邊界。層級由下往上編號：第一層是基礎，第五層是治理表面。



3.3 — Layer 1: Trust Source (OpenRNG Core)

3.3 — 第一層：信任來源（OpenRNG 核心）

The Trust Source layer is the foundation of the entire stack. Its responsibility is singular: produce high-quality, multi-source entropy as the raw material of trust. Everything above this layer — authorization, evidence, coordination, governance — depends on the quality and integrity of what this layer produces.

The layer operates through a four-stage pipeline:

Stage 1: Source Collection. The engine queries multiple independent entropy sources in parallel. Current production sources include drand (a distributed randomness beacon operated by the League of Entropy, including Cloudflare, Protocol Labs, and academic institutions), Bitcoin block hashes (proof-of-work entropy derived from the computational effort of the entire Bitcoin mining network), and Polygon block hashes (proof-of-stake entropy from validator consensus). The architecture is source-agnostic: future sources include Quantum Random Number Generators (QRNG), additional randomness beacons, and federated source networks.

Stage 2: Source Verification. Each source response is independently verified before inclusion. drand beacon signatures are checked against the published League of Entropy public key. Blockchain hashes are verified against on-chain state. Sources that fail verification are flagged but do not halt generation — they reduce the Entropy Confidence Score, providing transparent degradation rather than opaque failure.

Stage 3: Entropy Aggregation. Verified source entropies are combined using a deterministic aggregation function. The current implementation uses concatenation followed by SHA-256 hashing, producing a single entropy value that incorporates contributions from all verified sources. The aggregation method is recorded in the output for auditability.

Stage 4: Quality Scoring. The Entropy Confidence Score (ECS) is computed across six independently measured dimensions: freshness (time since generation), diversity (number of unique source identifiers), independence (number of unique source types), manipulation resistance (difficulty of controlling the output), verification success (proportion of sources that passed verification), and availability (source accessibility at generation time). The composite score ranges from 0 to 1000 with defined grade boundaries from AAA (≥ 900) to LOW (< 500).

The Trust Source layer also implements batch generation via Merkle trees — a single VDF (Verifiable Delay Function) seed can deterministically generate up to 65,536 entropy values organized in a Merkle tree, enabling sub-2ms serving latency while maintaining cryptographic proof for each individual value. This batch architecture is critical for production-scale deployment where latency budgets are measured in milliseconds.

信任來源層是整個堆疊的基礎。它的職責是單一的：產生高品質、多來源的熵作為信任的原始材料。此層之上的一切——授權、證據、協調、治理——都取決於此層產出的品質和完整性。

此層透過四階段管線運作：

階段一：來源收集。引擎平行查詢多個獨立熵源。當前生產環境來源包括 drand（由 Cloudflare、Protocol Labs 和學術機構等組成的 League of Entropy 運營的分散式隨機信標）、比特幣區塊雜湊（來自整個比特幣挖礦網路計算努力的工作量證明熵），以及 Polygon 區塊雜湊（來自驗證者共識的權益證明熵）。架構是來源無關的：未來來源包括量子隨機數生成器（QRNG）、額外的隨機信標和聯邦來源網路。

階段二：來源驗證。每個來源回應在納入前都會獨立驗證。drand 信標簽名會對照已公開的 League of Entropy 公鑰進行檢查。區塊鏈雜湊會對照鏈上狀態進行驗證。未通過驗證的來源會被標記，但不會中斷生成——它們會降低熵信心分數，提供透明的降級而非不透明的失敗。

階段三：熵聚合。經驗證的來源熵使用確定性聚合函數組合。當前實作使用串聯後進行 SHA-256 雜湊，產生包含所有已驗證來源貢獻的單一熵值。聚合方法記錄在輸出中以供稽核。

階段四：品質評分。熵信心分數（ECS）跨六個獨立量測的維度計算：新鮮度（生成後經過的時間）、多樣性（唯一來源識別碼數量）、獨立性（唯一來源類型數量）、抗操縱性（控制輸出的難度）、驗證成功率（通過驗證的來源比例），以及可用性（生成時的來源可存取性）。綜合分數範圍從 0 到 1000，定義的等級邊界從 AAA（ ≥ 900 ）到 LOW（ < 500 ）。

信任來源層還透過 Merkle 樹實作批次生成——單一 VDF（可驗證延遲函數）種子可以確定性地生成多達 65,536 個熵值，組織在 Merkle 樹中，實現低於 2 毫秒的服務延遲，同時為每個個別值維持密碼學證明。這種批次架構對於延遲預算以毫秒計算的生產規模部署至關重要。

Property	Specification
Purpose 目的	Generate high-quality, multi-source entropy 產生高品質、多來源熵
Input 輸入	Independent entropy sources (drand, Bitcoin, Polygon, QRNG) 獨立熵源
Output 輸出	Raw scored entropy with full source provenance 具完整來源出處的原始評分熵
Patent 專利	Patent #1 — Method and System for Gaming RNG 專利一號——遊戲隨機數生成方法與系統
Performance 性能	Sub-2ms serving via Merkle batch pre-generation 透過 Merkle 批次預生成實現低於 2ms 服務

3.4 — Layer 2: Trust Authorization (Lockbox)

3.4 — 第二層：信任授權（Lockbox）

The Trust Authorization layer — codenamed Lockbox — controls who may consume trust primitives and under what conditions. This is not traditional access control. Traditional access control (OAuth, RBAC, API keys) answers a single question: "Is this identity allowed to access this resource?" Trust Authorization answers a different question: "Is this consumer allowed to use this trust primitive, and can we prove they used it correctly?"

The distinction is attestation. OAuth grants access and forgets. Lockbox grants access and records cryptographic proof of correct usage. Every consumption event is attested — creating an irrefutable record that a specific consumer used a specific trust primitive at a specific time under specific policy constraints.

Lockbox enforces three categories of policy:

Consumption Policies: Rate limiting (maximum entropy requests per time window), quality floors (minimum ECS grade required for the consumer's use case), source constraints (which entropy source types are permitted or excluded), and anchoring requirements (whether blockchain anchoring is mandatory).

Delegation Policies: Whether a consumer can delegate their trust authorization to sub-agents, under what conditions, and with what constraints. This is critical for multi-agent AI systems where

a primary agent spawns child agents — the child must inherit authorization within defined boundaries.

Compliance Policies: Regulatory-context-specific constraints that map to frameworks like GLI-19 (gaming), EU AI Act (AI systems), or HIPAA (healthcare). These policies ensure that trust primitives consumed in regulated contexts meet the appropriate requirements automatically, without requiring the consumer to understand the regulatory details.

信任授權層——代號 Lockbox——控制誰可以消費信任原語，以及在什麼條件下消費。這不是傳統的存取控制。傳統存取控制（OAuth、RBAC、API 金鑰）回答一個問題：「此身份是否被允許存取此資源？」信任授權回答不同的問題：「此消費者是否被允許使用此信任原語，我們能否證明他們正確使用了它？」

區別在於證明。OAuth 授予存取權然後遺忘。Lockbox 授予存取權並記錄正確使用的密碼學證明。每個消費事件都被證明——建立不可否認的記錄：特定消費者在特定時間、特定策略約束下使用了特定信任原語。

Lockbox 執行三類策略：

消費策略：速率限制（每時間窗口最大熵請求數）、品質下限（消費者使用案例所需的最低 ECS 等級）、來源約束（允許或排除哪些熵源類型），以及錨定需求（是否強制要求區塊鏈錨定）。

委託策略：消費者是否可以將信任授權委託給子代理、在什麼條件下、以及有什麼約束。這對多代理 AI 系統至關重要——當主要代理產生子代理時，子代理必須在定義的邊界內繼承授權。

合規策略：對應到 GLI-19（遊戲）、EU AI Act（AI 系統）或 HIPAA（醫療保健）等框架的法規環境特定約束。這些策略確保在受監管環境中消費的信任原語自動滿足適當的要求，無需消費者理解法規細節。

Property	Specification
Purpose 目的	Policy-gated access with usage attestation 具使用證明的策略閘門存取
Input 輸入	Consumer identity, policy rules, trust requirements 消費者身份、策略規則、信任需求
Output 輸出	Authorization token with policy compliance proof 附策略合規證明的授權令牌
Patent 專利	Patent #2 — Lockbox (Trust Authorization) 專利二號——Lockbox（信任授權）
Key Innovation 關鍵創新	Attestation-based consumption — every use is provably recorded 基於證明的消費——每次使用都可證明地記錄

3.5 — Layer 3: Trust Evidence (VEO)

3.5 — 第三層：信任證據（VEO）

The Trust Evidence layer is the heart of the stack — the layer that transforms raw entropy into a standardized, self-describing, independently verifiable digital object. This layer produces the Verifiable Entropy Object (VEO-1), the core primitive of Computational Trust Infrastructure.

A VEO is not a random number with metadata. It is a **proof-carrying object** — a cryptographic container that enables any party, anywhere, at any time, to independently verify the integrity, provenance, quality, and authenticity of the entropy it carries, without trusting the issuer or contacting any external service.

The VEO-1 specification defines four object classes: VEO-1A (single-source raw entropy), VEO-1B (multi-source composite entropy), VEO-1C (blockchain-anchored entropy with Merkle proof or transaction hash), and VEO-1D (reserved for Verifiable Decision Objects — entropy objects extended with decision context). Each class represents a different level of trust evidence.

Every VEO carries five layers of information:

Payload: The entropy value itself, plus its SHA-256 hash. Any verifier can recompute the hash from the payload to confirm integrity.

Provenance: An array of source records, each containing source identifier, source type, reference (e.g., "drand-round-29830144"), timestamp, and per-source entropy hash. This creates a

complete audit trail of where every bit of entropy originated.

Confidence: The Entropy Confidence Score (ECS) — a composite score (0–1000) computed across six weighted dimensions, with a letter grade from AAA to LOW. The score is deterministically reproducible: given the same source records, any party can independently recompute the same ECS.

Proof: A secp256k1 EIP-191 provider signature over the canonicalized VEO payload. Canonicalization uses deep key sorting and deterministic serialization to ensure that the same VEO always produces the same signing payload, regardless of JSON key ordering. The signature can be verified against the provider's public key without any network access.

Anchor (optional): A blockchain anchor — either a direct transaction hash or a Merkle proof linking the VEO to an on-chain Merkle root. The Merkle batch system enables a single on-chain transaction to anchor up to 65,536 VEOs, providing temporal proof (the VEO existed at or before the block timestamp) and immutability (the anchor cannot be altered after confirmation).

The VEO-1 verification framework defines five levels:

信任證據層是堆疊的核心——將原始熵轉化為標準化、自描述、可獨立驗證的數位物件的層。此層產生可驗證熵物件（VEO-1），計算信任基礎設施的核心原語。

VEO 不是附帶元數據的隨機數。它是一個**攜帶證明的物件**——一個密碼學容器，使任何一方、在任何地方、在任何時間，都能獨立驗證其攜帶的熵的完整性、出處、品質和真實性，無需信任發行者或聯繫任何外部服務。

VEO-1 規範定義了四個物件類別：VEO-1A（單來源原始熵）、VEO-1B（多來源複合熵）、VEO-1C（具 Merkle 證明或交易雜湊的區塊鏈錨定熵），以及 VEO-1D（保留給可驗證決策物件——擴展了決策上下文的熵物件）。每個類別代表不同層級的信任證據。

每個 VEO 攜帶五層資訊：

載荷：熵值本身，加上其 SHA-256 雜湊。任何驗證者都可以從載荷重新計算雜湊以確認完整性。

出處：一組來源記錄，每條包含來源識別碼、來源類型、參考（如「drand-round-29830144」）、時間戳和每來源熵雜湊。這為每一位元熵的起源建立了完整的稽核軌跡。

信心：熵信心分數（ECS）——跨六個加權維度計算的綜合分數（0-1000），附字母等級從 AAA 到 LOW。分數是確定性可重現的：給定相同的來源記錄，任何一方都可以獨立重新計算相同的 ECS。

證明：對規範化 VEO 載荷的 secp256k1 EIP-191 供應商簽名。規範化使用深層鍵排序和確定性序列化，確保相同的 VEO 總是產生相同的簽名載荷，無論 JSON 鍵的排序如何。簽名可以在沒有任何網路存取的情況下對照供應商的公鑰進行驗證。

錨定（可選）：區塊鏈錨定——直接交易雜湊或將 VEO 連結到鏈上 Merkle 根的 Merkle 證明。Merkle 批次系統使單一鏈上交易能夠錨定多達 65,536 個 VEO，提供時間證明（VEO 在區塊時間戳之前或之時存在）和不可變性（錨定在確認後無法更改）。

VEO-1 驗證框架定義了五個層級：

Level 層級	Name 名稱	Requirements 要求
1	<code>structurally_valid_unsigned</code>	Entropy hash matches payload; no signature required 熵雜湊匹配載荷；不需要簽名
2	<code>cryptographically_verified</code>	Hash valid + provider signature verified against public key 雜湊有效 + 供應商簽名對照公鑰驗證通過
3	<code>anchored_verified</code>	All above + Merkle proof valid against on-chain root 以上全部 + Merkle 證明對照鏈上根有效
4	<code>policy_failed</code>	Object valid but ECS or source constraints not met 物件有效但 ECS 或來源約束未滿足
5	<code>invalid</code>	Hash mismatch, signature invalid, or structural error 雜湊不匹配、簽名無效或結構錯誤

Property	Specification
Purpose 目的	Package trust into standardized, verifiable objects 將信任封裝為標準化、可驗證的物件
Input 輸入	Scored entropy + authorization proof + consumer policy 已評分熵 + 授權證明 + 消費者策略
Output 輸出	VEO-1 — self-describing, signed, anchorable trust evidence VEO-1 — 自描述、已簽名、可錨定的信任證據
Patent 專利	Patent #3 — VEO (Trust Evidence) 專利三號 — VEO (信任證據)
Standard 標準	VEO-1 v1.0 (Frozen), ECS v1 (Frozen), RFC-0001 VEO-1 v1.0 (已凍結)、ECS v1 (已凍結)、RFC-0001

3.6 — Layer 4: Trust Coordination

3.6 — 第四層：信任協調

Layers 1 through 3 solve the trust problem for individual computations: a single agent, a single decision, a single verifiable object. But modern AI systems are not single-agent. They are swarms — dozens or hundreds of autonomous agents collaborating, delegating, and handing off tasks. When Agent A assigns a task to Agent B based on verifiable entropy, Agent B needs to trust that the assignment was fair — and Agent C (the auditor) needs to verify the entire chain.

The Trust Coordination layer solves the **horizontal trust propagation problem**: how does trust flow between autonomous agents and across system boundaries?

Trust Chain Construction. When a VEO is used to make a decision that spawns child computations, the child VEOs reference the parent via the lineage field. This creates a directed acyclic graph (DAG) of trust — every decision can be traced back to its root entropy. A task allocation VEO spawns sub-task VEOs, which spawn tool-selection VEOs, forming a complete, auditable decision tree.

Multi-Agent Trust Consensus. When multiple agents must agree on a shared random value — for example, jointly selecting a leader, allocating a contested resource, or establishing turn order — Trust Coordination provides a protocol for combining individual VEOs into a consensus VEO. Each participant contributes entropy; the consensus VEO references all contributions; any party can verify that the consensus was computed correctly from the individual inputs.

Cross-System Trust Handoff. When trust needs to cross organizational boundaries — an enterprise AI system delegating to a third-party agent service — Trust Coordination defines the protocol for packaging trust assertions, transmitting them across trust boundaries, and verifying them on the receiving side. The handoff protocol ensures that trust does not require mutual trust between organizations — only trust in the cryptographic proofs.

Trust Propagation Rules. Three invariants govern trust propagation: (1) Trust does not amplify — a child VEO's ECS cannot exceed its parent's; (2) Trust degrades transparently — if a child uses fewer sources, the degradation is recorded, not hidden; (3) Trust is traceable — every VEO in a chain can be independently verified without trusting intermediate nodes.

It is critical to understand what Trust Coordination is *not*. It is not an orchestrator. It does not control agents or decide what they do. It is a **protocol for trust transmission** — analogous to TCP for data transmission. TCP does not decide what data to send; it ensures data arrives correctly. Trust Coordination does not decide what agents do; it ensures trust propagates correctly.

第一到三層解決的是個別計算的信任問題：單一代理、單一決策、單一可驗證物件。但現代 AI 系統不是單一代理的。它們是群體——數十或數百個自主代理協作、委託和交接任務。當代理 A 基於可驗證熵將任務分配給代理 B 時，代理 B 需要信任分配是公平的——而代理 C（稽核員）需要驗證整條鏈。

信任協調層解決的是**水平信任傳播問題**：信任如何在自主代理之間和跨系統邊界流動？

信任鏈建構。當 VEO 被用於做出產生子計算的決策時，子 VEO 透過血統欄位參考父代。這建立了信任的有向無環圖（DAG）——每個決策都可以追溯到其根熵。任務分配 VEO 產生子任務 VEO，再產生工具選擇 VEO，形成完整、可稽核的決策樹。

多代理信任共識。當多個代理必須就共享隨機值達成一致——例如共同選舉領導者、分配爭議資源或確定順序——信任協調提供將個別 VEO 組合為共識 VEO 的協議。每個參與者貢獻熵；共識 VEO 參考所有貢獻；任何一方都可以驗證共識是否從個別輸入正確計算。

跨系統信任交接。當信任需要跨越組織邊界——企業 AI 系統委託給第三方代理服務——信任協調定義了封裝信任斷言、跨越信任邊界傳輸、並在接收端驗證的協議。交接協議確保信任不需要組織間的相互信任——只需要信任密碼學證明。

信任傳播規則。三個不變量治理信任傳播：(1) 信任不會放大——子 VEO 的 ECS 不能超過其父代；(2) 信任透明降級——如果子代使用更少的來源，降級被記錄而非隱藏；(3) 信任可追溯——鏈中的每個 VEO 都可以在不信任中間節點的情況下獨立驗證。

理解信任協調**不是什麼**至關重要。它不是編排器。它不控制代理或決定它們做什麼。它是**信任傳輸協議**——類似於 TCP 之於資料傳輸。TCP 不決定發送什麼資料；它確保資料正確到達。信任協調不決定代理做什麼；它確保信任正確傳播。

Property	Specification
Purpose 目的	Enable trust to flow between agents and across boundaries 使信任在代理之間和跨邊界流動
Input 輸入	VEOs from multiple participants, coordination policies 來自多個參與者的 VEO、協調策略
Output 輸出	Trust chains, consensus VEOs, cross-boundary handoff proofs 信任鏈、共識 VEO、跨邊界交接證明
Patent 專利	Patent #4 — Trust Coordination (New Filing) 專利四號——信任協調（新申請）
Direction 方向	Horizontal — peer-to-peer trust propagation 水平——點對點信任傳播

3.7 — Layer 5: Trust Runtime

3.7 — 第五層：信任運行時

The Trust Runtime layer provides governance, compliance, and continuous verification for the entire trust infrastructure. While Layer 2 (Trust Authorization) enforces policy at the point of consumption — a per-request gate — the Trust Runtime enforces policy across the entire lifecycle, retroactively, continuously, and against evolving requirements.

Runtime Policy Enforcement. The Trust Runtime monitors and enforces system-wide policies that cannot be evaluated at the per-request level: maximum trust chain depth (e.g., no VEO lineage deeper than five levels), source rotation requirements (e.g., no single source may dominate more than 70% of requests over any 24-hour window), anchoring frequency (e.g., all VEOs above a certain value threshold must be anchored within 60 seconds), and temporal consistency (e.g., no VEO timestamps may be out of sequence).

Compliance Attestation. The Trust Runtime generates structured compliance attestations that map trust infrastructure operations to regulatory framework requirements. For the EU AI Act, the runtime demonstrates that AI decisions backed by VEOs satisfy Article 14 (Human Oversight) transparency requirements. For NIST AI RMF, the trust lifecycle maps directly to the GOVERN function. For GLI-19 (Gaming Laboratories International), VEO verification satisfies RNG certification requirements. These attestations are machine-readable and designed for integration with enterprise compliance systems.

Trust Health Monitoring. The runtime continuously monitors trust infrastructure health metrics: system-wide average ECS over time (detecting source degradation), source availability and diversity trends, verification success rates, mean time between anchoring, trust chain depth distribution, and consumption patterns. Anomalies trigger alerts — for example, a sustained decline in average ECS may indicate that an entropy source is degrading, requiring investigation before it affects downstream trust quality.

The boundary between Layer 4 (Trust Coordination) and Layer 5 (Trust Runtime) is precise and intentional: **Coordination is horizontal** — it governs how trust flows between peers. **Runtime is vertical** — it governs how trust is enforced, monitored, and attested from top to bottom. Coordination is a protocol. Runtime is a governance engine.

信任運行時層為整個信任基礎設施提供治理、合規和持續驗證。第二層（信任授權）在消費點執行策略——逐請求的閘門——而信任運行時跨越整個生命週期執行策略，具有追溯性、持續性，並適應不斷演變的要求。

運行時策略執行。信任運行時監控並執行無法在逐請求層級評估的系統範圍策略：最大信任鏈深度（如 VEO 血統不超過五層）、來源輪換要求（如任何 24 小時窗口內單一來源不得主導超過 70% 的請求）、錨定頻率（如超過特定價值閾值的所有 VEO 必須在 60 秒內錨定），以及時間一致性（如 VEO 時間戳不得亂序）。

合規證明。信任運行時生成結構化的合規證明，將信任基礎設施操作對應到法規框架要求。對於 EU AI Act，運行時證明由 VEO 支持的 AI 決策滿足第 14 條（人類監督）透明度要求。對於 NIST AI RMF，信任生命週期直接對應到 GOVERN 功能。對於 GLI-19（遊戲實驗室國際），VEO 驗證滿足 RNG 認證要求。這些證明是機器可讀的，設計用於與企業合規系統整合。

信任健康監控。運行時持續監控信任基礎設施健康指標：系統範圍的平均 ECS 隨時間變化（偵測來源降級）、來源可用性和多樣性趨勢、驗證成功率、平均錨定間隔時間、信任鏈深度分佈，以及消費模式。異常觸發警報——例如，平均 ECS 的持續下降可能表明某個熵源正在降級，需要在影響下游信任品質之前進行調查。

第四層（信任協調）和第五層（信任運行時）之間的邊界是精確且有意的：協調是水平的——治理信任如何在對等方之間流動。運行時是垂直的——治理信任如何從上到下被執行、監控和證明。協調是協議。運行時是治理引擎。

Property	Specification
Purpose 目的	Governance, compliance, and continuous verification 治理、合規和持續驗證
Input 輸入	Trust stack telemetry, regulatory requirements, org policies 信任堆疊遙測、法規要求、組織策略
Output 輸出	Compliance attestations, health metrics, governance reports 合規證明、健康指標、治理報告
Patent 專利	Patent #5 — Trust Runtime (New Filing) 專利五號——信任運行時（新申請）
Direction 方向	Vertical — policy enforcement and audit from top to bottom 垂直——從上到下的策略執行和稽核

3.8 — Cross-Cutting Concerns

3.8 — 跨層關注事項

Versioning and Backward Compatibility. The Trust Stack follows a strict versioning policy. The VEO-1 specification and ECS v1 scoring algorithm are frozen — required fields, dimension weights, and grade boundaries cannot change without a protocol version bump. New optional fields may be added in minor versions. This stability is critical for trust: a VEO issued today must be verifiable ten years from now using the same verification logic.

Performance. Trust infrastructure must not impose unacceptable latency. The Trust Source layer achieves sub-2ms serving through Merkle batch pre-generation — entropy values are generated ahead of demand and served from a pre-computed pool. The Evidence layer adds approximately 1ms for signing. Blockchain anchoring is asynchronous and does not block the critical path. The total overhead for a Standard-profile interaction (Require → Generate → Authorize → Execute → Evidence) is under 5ms — negligible for any production use case.

Failure Modes. The Trust Stack is designed for transparent degradation. When an entropy source is unavailable, the system continues with available sources and reduces the ECS score accordingly. When blockchain anchoring fails, the VEO is issued at a lower verification level (cryptographically_verified instead of anchored_verified) with the failure reason recorded. There are no silent failures — every degradation is visible in the VEO's confidence and proof fields.

Extensibility. Each layer is designed for extension without modification. New entropy sources can be added to Layer 1 by implementing a source adapter. New authorization policies can be added to Layer 2 through the policy engine. New verification types can be added to Layer 3 by extending the proof package. The five-layer structure itself is stable; what changes is the implementation within each layer.

版本控制和向後相容性。信任堆疊遵循嚴格的版本控制策略。VEO-1 規範和 ECS v1 評分演算法已凍結——必填欄位、維度權重和等級邊界在沒有協議版本升級的情況下不能更改。新的可選欄位可以在次要版本中添加。這種穩定性對信任至關重要：今天發行的 VEO 必須在十年後使用相同的驗證邏輯仍可驗證。

性能。信任基礎設施不得施加不可接受的延遲。信任來源層透過 Merkle 批次預生成實現低於 2ms 的服務——熵值在需求之前生成，並從預計算池中提供。證據層為簽名增加約 1ms。區塊鏈錨定是非同步的，不阻塞關鍵路徑。標準配置互動（需求 → 生成 → 授權 → 執行 → 證據）的總開銷低於 5ms——對任何生產使用案例來說可以忽略不計。

故障模式。信任堆疊設計為透明降級。當熵源不可用時，系統繼續使用可用來源並相應降低 ECS 分數。當區塊鏈錨定失敗時，VEO 以較低的驗證層級發行（cryptographically_verified 而非 anchored_verified），並記錄失敗原因。沒有靜默失敗——每次降級都在 VEO 的信心和證明欄位中可見。

可擴展性。每一層都設計為可擴展而無需修改。新的熵源可以透過實作來源適配器添加到第一層。新的授權策略可以透過策略引擎添加到第二層。新的驗證類型可以透過擴展證明包添加到第三層。五層結構本身是穩定的；變化的是每層內的實作。

The Trust Lifecycle

信任生命週期

The operational heartbeat of Computational Trust Infrastructure

計算信任基礎設施的運行核心

4.1 — Why a Lifecycle?

4.1 — 為什麼需要生命週期？

Every infrastructure category has a lifecycle — a canonical sequence of operations that defines how the infrastructure is used, not just what it is.

TLS has a lifecycle: handshake → key exchange → encrypted session → termination. OAuth has a lifecycle: authorization request → user consent → token issuance → resource access → token refresh → revocation. DNS has a lifecycle: query → recursive resolution → caching → TTL expiry → re-resolution.

These lifecycles are what make infrastructure *operational*. Without them, a technology is just a specification. With them, it becomes something engineers build systems around.

Computational Trust Infrastructure is no different. The Trust Stack (Chapter 3) defines the architecture — the layers, their responsibilities, their interfaces. The Trust Lifecycle defines how those layers operate in sequence to produce, propagate, and verify trust.

If the Trust Stack is the anatomy of the system, the Trust Lifecycle is its physiology.

每個基礎設施類別都有一個生命週期——定義基礎設施如何使用（而不僅僅是它是什麼）的規範操作序列。

TLS 有生命週期：握手 → 金鑰交換 → 加密會話 → 終止。OAuth 有生命週期：授權請求 → 使用者同意 → 令牌發行 → 資源存取 → 令牌刷新 → 撤銷。DNS 有生命週期：查詢 → 遞迴解析 → 快取 → TTL 過期 → 重新解析。

這些生命週期使基礎設施具有可操作性。沒有它們，技術只是規範。有了它們，它就成為工程師建構系統的依據。

計算信任基礎設施也不例外。信任堆疊（第三章）定義了架構——層級、其職責、其介面。信任生命週期定義了這些層如何依序運作來產生、傳播和驗證信任。

如果信任堆疊是系統的解剖學，信任生命週期就是其生理學。

4.2 — The Eight Stages

4.2 — 八個階段

The Trust Lifecycle consists of eight stages. Not every use case requires all eight. The lifecycle is designed to be **truncatable** — simpler use cases use fewer stages, and the system remains valid at every truncation point.

信任生命週期由八個階段組成。並非每個使用案例都需要全部八個階段。生命週期設計為可截斷的——較簡單的使用案例使用較少的階段，系統在每個截斷點都保持有效。

THE TRUST LIFECYCLE



Each stage transforms a trust state. Each transition produces a verifiable artifact. The lifecycle is not a pipeline — it is a state machine with defined invariants at every stage.

每個階段轉換一個信任狀態。每次轉換都產生一個可驗證的成品。生命週期不是管線——它是在每個階段都有定義不變量的狀態機。

Stage 1: REQUIRE — 需求

What trust does this computation need?

A system — an AI agent, a game server, a clinical trial platform — encounters a decision point that requires non-deterministic input. Before generating anything, the system must articulate what kind of trust it needs: quality threshold (minimum ECS grade), source constraints (which entropy sources are acceptable), verification level (structural, cryptographic, or anchored), latency budget, anchoring requirements, and regulatory context.

The output is a **Trust Requirement Object (TRO)** — a structured declaration of what the consumer needs. The TRO is immutable once issued; if requirements change, a new TRO must be created, ensuring audit trails can always trace back to the original requirement.

此計算需要什麼信任？

一個系統——AI 代理、遊戲伺服器、臨床試驗平台——遇到需要非確定性輸入的決策點。在生成任何東西之前，系統必須清楚表達它需要什麼類型的信任：品質閾值（最低 ECS 等級）、來源約束（可接受哪些熵源）、驗證層級（結構性、密碼學或錨定）、延遲預算、錨定要求和法規背景。

輸出是**信任需求物件（TRO）**——消費者需求的結構化聲明。TRO 一旦發出即不可變；如果需求變更，必須建立新的 TRO，確保稽核軌跡始終可以追溯到原始需求。

Stage 2: GENERATE — 生成

Where does the raw trust material come from?

The Trust Source layer (Layer 1) produces raw entropy through a four-stage pipeline: source collection (querying multiple independent entropy sources in parallel), source verification (independently verifying each response — drand signatures, blockchain hashes), entropy aggregation (deterministic combination via concatenation + SHA-256), and quality scoring (computing the six-dimension ECS). The generation process is deterministic given the same source inputs — any auditor with access to the source records can re-derive the aggregated entropy and verify the ECS score.

原始信任材料從哪裡來？

信任來源層（第一層）透過四階段管線產生原始熵：來源收集（平行查詢多個獨立熵源）、來源驗證（獨立驗證每個回應——drand 簽名、區塊鏈雜湊）、熵聚合（透過串聯 + SHA-256 的確定性組合），以及品質評分（計算六維度 ECS）。生成過程在給定相同來源輸入時是確定性的——任何有權存取來源記錄的稽核員都可以重新導出聚合熵並驗證 ECS 分數。

Stage 3: AUTHORIZE — 授權

Is this consumer allowed to use this trust primitive, under these conditions?

The Trust Authorization layer (Layer 2 — Lockbox) enforces policy on consumption. Unlike traditional access control, Trust Authorization produces attestation — cryptographic proof that a specific consumer used a specific trust primitive at a specific time under specific policy constraints. Every consumption event is attested. There is no "anonymous" entropy consumption in a trust infrastructure.

此消費者是否被允許在這些條件下使用此信任原語？

信任授權層（第二層——Lockbox）對消費執行策略。與傳統存取控制不同，信任授權產生證明——特定消費者在特定時間、特定策略約束下使用了特定信任原語的密碼學證明。每個消費事件都被證明。在信任基礎設施中沒有「匿名」熵消費。

Stage 4: EXECUTE — 執行

What computation was performed using the trust primitive?

The consumer uses the authorized trust primitive in their computation — an AI agent selects a tool, a casino determines an outcome, a trial platform randomizes patients. This stage is intentionally outside the Trust Stack's direct control. The stack provides the primitive; the consumer provides the computation. This separation is critical for adoption: consumers should not need to restructure their applications to use verifiable entropy.

使用信任原語執行了什麼計算？

消費者在其計算中使用授權的信任原語——AI 代理選擇工具、賭場確定結果、試驗平台隨機分配患者。此階段有意置於信任堆疊的直接控制之外。堆疊提供原語；消費者提供計算。這種分離對採用至關重要：消費者不應需要重組其應用程式來使用可驗證熵。

Stage 5: EVIDENCE — 證據

Can we package what happened into a verifiable, portable proof?

The Trust Evidence layer (Layer 3) transforms the computation into a VEO-1 object through four operations: object construction (assembling payload, sources, ECS), canonical signing (deterministic serialization + secp256k1 EIP-191 signature), blockchain anchoring (optional Merkle batch on Polygon), and verification level assignment. The resulting VEO is self-verifying: given only the object and the provider's public key, any party can verify integrity, authenticity, quality, and temporal anchoring — offline, air-gapped, years later.

我們能否將發生的事情封裝為可驗證、可攜帶的證明？

信任證據層（第三層）透過四個操作將計算轉化為 VEO-1 物件：物件建構（組裝載荷、來源、ECS）、規範簽名（確定性序列化 + secp256k1 EIP-191 簽名）、區塊鏈錨定（可選的 Polygon Merkle 批次），以及驗證層級指定。產出的 VEO 是自驗證的：僅憑物件和供應商的公鑰，任何一方都可以驗證完整性、真實性、品質和時間錨定——離線、氣隙隔離、多年之後。

Stage 6: COORDINATE — 協調

How does trust flow between multiple agents and systems?

In multi-agent AI systems, trust must propagate across autonomous agents. Trust Coordination enables trust chain construction (child VEOs reference parents via lineage, forming a DAG), multi-agent trust consensus (combining individual VEOs into a jointly verifiable consensus), and cross-system trust handoff (packaging trust assertions for transmission across organizational boundaries). Three invariants govern propagation: trust does not amplify, trust degrades transparently, and trust is always traceable.

信任如何在多個代理和系統之間流動？

在多代理 AI 系統中，信任必須跨自主代理傳播。信任協調實現信任鏈建構（子 VEO 透過血統參考父代，形成 DAG）、多代理信任共識（將個別 VEO 組合為可聯合驗證的共識），以及跨系統信任交接（封裝信任斷言以跨組織邊界傳輸）。三個不變量治理傳播：信任不會放大、信任透明降級、信任始終可追溯。

Stage 7: GOVERN — 治理

Does this trust chain comply with policy and regulation?

The Trust Runtime (Layer 5) provides continuous governance: runtime policy enforcement (trust chain depth limits, source rotation requirements, anchoring frequency), compliance attestation (mapping to EU AI Act, NIST AI RMF, GLI-19, ISO 42001), anomaly detection (ECS drift, source concentration, consumption pattern analysis), and trust health monitoring. Authorization (Stage 3) is a per-request gate; Governance is a continuous monitor across the entire lifecycle.

此信任鏈是否符合策略和法規？

信任運行時（第五層）提供持續治理：運行時策略執行（信任鏈深度限制、來源輪換要求、錨定頻率）、合規證明（對應 EU AI Act、NIST AI RMF、GLI-19、ISO 42001）、異常偵測（ECS 漂移、來源集中度、消費模式分析），以及信任健康監控。授權（階段三）是逐請求的閘門；治理是跨越整個生命週期的持續監控器。

Stage 8: AUDIT — 稽核

Can an independent third party verify everything that happened?

The reason the entire lifecycle exists. An auditor can verify any individual VEO (recompute hash, verify signature, reproduce ECS, validate Merkle proof), traverse trust chains (follow lineage from any VEO to its root), verify policy compliance (check every VEO against its TRO), and produce structured audit reports. The critical property: audit requires **zero trust in the operator**. Every verification is performed independently using cryptographic proofs and public blockchain data.

獨立第三方能否驗證所發生的一切？

整個生命週期存在的理由。稽核員可以驗證任何個別 VEO（重新計算雜湊、驗證簽名、重現 ECS、驗證 Merkle 證明）、遍歷信任鏈（從任何 VEO 跟隨血統到其根）、驗證策略合規（對照 TRO 檢查每個 VEO），並產生結構化稽核報告。關鍵特性：稽核不需要信任操作者。每個驗證都使用密碼學證明和公開的區塊鏈資料獨立執行。

4.3 — Lifecycle Profiles

4.3 — 生命週期配置

Not every use case requires all eight stages. The Trust Lifecycle supports four predefined profiles — truncations for common use cases.

並非每個使用案例都需要全部八個階段。信任生命週期支持四個預定義配置——針對常見使用案例的截斷。

Profile 配置	Stages 階段	Use Case 使用案例	VEO Level VEO 層級
Minimal 最小	Require → Generate → Evidence (3)	Development, testing, low-stakes 開發、測試、低風險	<code>structurally_valid_unsigned</code>
Standard 標準	Require → Generate → Authorize → Execute → Evidence (5)	Production AI agents, enterprise 生產 AI 代理、企業	<code>cryptographically_verified</code>
Regulated 受監管	Require → ... → Evidence → Audit (6)	Gaming, finance, healthcare 遊戲、金融、醫療	<code>anchored_verified</code>
Autonomous 自主	All 8 stages 全部八階段	Multi-agent swarms, cross-org AI 多代理群體、跨組織 AI	<code>anchored_verified</code>

Profiles are not restrictions — they are entry points. A consumer can start with Minimal and progressively adopt more stages as their trust requirements grow. The lifecycle is additive, not all-or-nothing.

配置不是限制——而是入口點。消費者可以從最小配置開始，隨著信任需求增長逐步採用更多階段。生命週期是遞增式的，不是全有或全無。

4.4 — Lifecycle Invariants

4.4 — 生命週期不變量

Regardless of which profile is used, the following invariants always hold:

1. **Every stage produces a verifiable artifact.** There are no "trust me" steps. If a stage was executed, its output can be independently verified.

2. **No stage requires trusting the previous stage's operator.** Each artifact carries its own proof. A Stage 5 VEO can be verified without trusting the Stage 2 authorizer or the Stage 1 generator.
3. **The lifecycle is re-entrant.** A VEO from one lifecycle instance can be an input to another. This enables composability: complex trust chains are built from simple, individually verifiable units.
4. **Partial lifecycles are valid.** Stopping at Stage 3 is a valid, complete interaction. The system does not require all stages to produce value.
5. **Trust does not amplify.** A child VEO cannot claim higher trust than its parent. Trust can only degrade or remain constant — never increase.
6. **Lifecycle metadata is append-only.** Once a stage's artifact is produced, it is immutable. Corrections require a new lifecycle instance.
7. **Time moves forward.** Each stage's timestamp must be equal to or later than the previous stage's. Temporal ordering is enforced and verifiable.

無論使用哪個配置，以下不變量始終成立：

1. **每個階段都產生可驗證的成品。**沒有「請相信我」的步驟。如果一個階段被執行了，其輸出可以被獨立驗證。
2. **沒有階段需要信任前一階段的操作者。**每個成品攜帶自己的證明。第五階段的 VEO 可以在不信任第二階段授權者或第一階段生成者的情況下驗證。
3. **生命週期是可重入的。**一個生命週期實例的 VEO 可以成為另一個的輸入。這實現了可組合性：複雜的信任鏈由簡單的、可個別驗證的單元構建。
4. **部分生命週期是有效的。**在第三階段停止是有效的、完整的互動。系統不需要所有階段來產生價值。
5. **信任不會放大。**子 VEO 不能聲稱比其父代更高的信任。信任只能降級或保持恆定——永遠不會增加。
6. **生命週期元數據是僅可追加的。**一旦階段的成品產出，它就是不可變的。更正需要新的生命週期實例。
7. **時間向前移動。**每個階段的時間戳必須等於或晚於前一階段的。時間順序被強制執行且可驗證。

4.5 — The Lifecycle as Category-Defining Primitive

4.5 — 生命週期作為類別定義原語

The Trust Lifecycle is not a feature of OpenRNG. It is the defining characteristic of the Computational Trust category.

信任生命週期不是 OpenRNG 的一個功能。它是計算信任類別的定義性特徵。

Infrastructure 基礎設施	Defining Lifecycle 定義性生命週期
TLS	Handshake → Key Exchange → Encrypted Session → Termination 握手 → 金鑰交換 → 加密會話 → 終止
OAuth	Auth Request → Consent → Token → Access → Refresh → Revoke 授權請求 → 同意 → 令牌 → 存取 → 刷新 → 撤銷
DNS	Query → Resolution → Cache → TTL Expiry → Re-resolve 查詢 → 解析 → 快取 → TTL 過期 → 重新解析
Computational Trust 計算信任	Require → Generate → Authorize → Execute → Evidence → Coordinate → Govern → Audit 需求 → 生成 → 授權 → 執行 → 證據 → 協調 → 治理 → 稽核

The goal: in five years, developers should not think about the Trust Lifecycle consciously. They should just know that when they call `openrng.getEntropy({ policy: 'ai-grade' })`, the lifecycle is executing — from requirement to evidence to audit readiness — invisibly, reliably, and verifiably.

The lifecycle is what transforms OpenRNG from a product into infrastructure.

目標：五年後，開發者不應有意識地思考信任生命週期。他們只需要知道，當他們呼叫 `openrng.getEntropy({ policy: 'ai-grade' })` 時，生命週期正在執行——從需求到證據到稽核就緒——無形地、可靠地、可驗證地。

生命週期是將 OpenRNG 從產品轉化為基礎設施的關鍵。

PART III

The Market

市場

The Patent Family

專利家族

Five patents, one architecture

五件專利，一套架構

5.1 — Family Overview

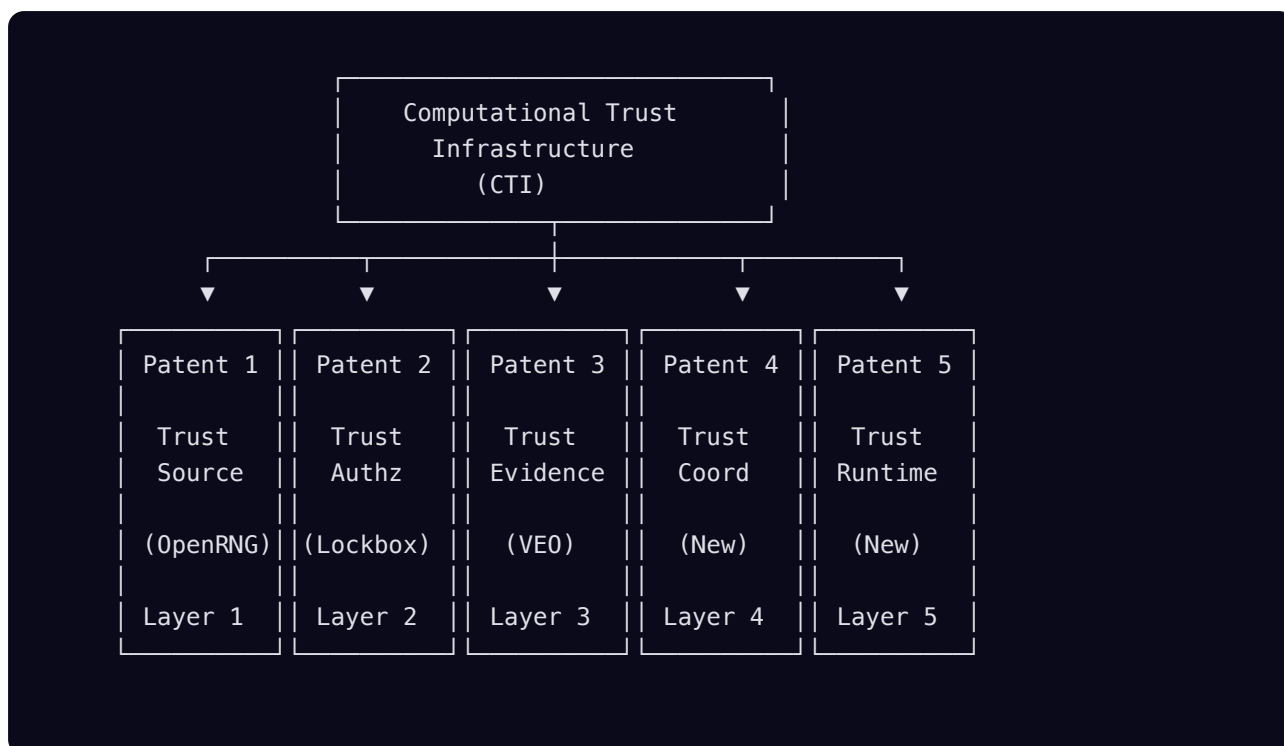
5.1 — 家族概覽

The OpenRNG patent family is not a collection of independent inventions. It is a coordinated set of claims that, together, cover the complete Computational Trust Infrastructure — from entropy generation through governance. Each patent maps to exactly one layer of the Trust Stack. Each claims a distinct responsibility. Together, they form a defensible, non-overlapping intellectual property portfolio that mirrors the architecture it protects.

The analogy is Apple's chip family: the A-series, M-series, S-series, and U-series are not independent products — they are components of a unified computing platform, each optimized for its layer (mobile, desktop, wearable, spatial). Similarly, the OpenRNG patent family comprises five components of a unified trust platform.

OpenRNG 專利家族不是獨立發明的集合。它是一組協調的權利主張，共同涵蓋完整的計算信任基礎設施——從熵生成到治理。每件專利精確對應信任堆疊的一層。每件主張一個獨特的職責。它們共同形成一個可防禦的、不重疊的智慧財產權組合，與它所保護的架構相互映照。

類比是蘋果的晶片家族：A 系列、M 系列、S 系列和 U 系列不是獨立產品——它們是統一運算平台的組件，每個都為其層級（行動、桌面、穿戴、空間）最佳化。同樣，OpenRNG 專利家族包含統一信任平台的五個組件。



5.2 — Patent #1: Trust Source Layer (OpenRNG Core)

5.2 — 專利一號：信任來源層（OpenRNG 核心）

Original Filing: "Method and System for Gaming Random Number Generation"

New Positioning: The foundational Trust Source — the layer that produces high-quality, multi-source entropy as the raw material of Computational Trust.

Key Claims:

- Verifiable Delay Function (VDF) seed generation for deterministic, time-locked entropy production
- Deterministic batch generation: VDF seed → N leaf hashes organized in a Merkle tree
- On-chain Merkle root anchoring via a single blockchain transaction per batch (up to 65,536 values)
- Pool-based serving with sub-2ms latency: pre-generated entropy served from memory with Merkle proof inclusion
- Multi-source entropy aggregation from independent providers (distributed beacons, blockchain hashes, hardware RNG)

Layer Mapping: Layer 1 of the Trust Stack

Technical Moat: The combination of VDF (time-locked, non-parallelizable computation), Merkle batch generation (one transaction proves thousands of values), and pool serving (sub-2ms latency) creates a unique architecture that is not covered by existing patents. Chainlink's VRF patent (US12401532B2) covers distributed oracle-based VRF; OpenRNG uses centralized VDF + Merkle batch — a structurally different architecture, different cryptographic primitive, and different trust model.

原始申請：「遊戲隨機數生成方法與系統」

新定位：基礎性的信任來源——產生高品質、多來源熵作為計算信任原始材料的層。

關鍵權利主張：

- 可驗證延遲函數（VDF）種子生成，用於確定性、時間鎖定的熵生產
- 確定性批次生成：VDF 種子 → N 個葉子雜湊組織在 Merkle 樹中
- 透過每批次單一區塊鏈交易進行鏈上 Merkle 根錨定（最多 65,536 個值）
- 基於池的服務，低於 2ms 延遲：從記憶體中提供預生成的熵，附帶 Merkle 證明
- 來自獨立供應商的多來源熵聚合（分散式信標、區塊鏈雜湊、硬體 RNG）

層級對應：信任堆疊的第一層

技術護城河：VDF（時間鎖定、不可並行化計算）、Merkle 批次生成（一筆交易證明數千個值）和池服務（低於 2ms 延遲）的組合建立了獨特的架構，不被現有專利涵蓋。Chainlink 的 VRF 專利（US12401532B2）涵蓋基於分散式預言機的 VRF；OpenRNG 使用集中式 VDF + Merkle 批次——結構上不同的架構、不同的密碼學原語和不同的信任模型。

5.3 — Patent #2: Trust Authorization Layer (Lockbox)

5.3 — 專利二號：信任授權層（Lockbox）

New Positioning: Policy-gated access to trust primitives with cryptographic usage attestation.

Key Claims:

- Policy engine for entropy consumption that evaluates requests against multi-dimensional constraints (quality floors, source requirements, rate limits, regulatory context)
- Cryptographic usage attestation: every consumption event produces an immutable, signed record proving that a specific consumer used a specific trust primitive under specific policy constraints

- Delegated authorization for multi-agent systems: a primary agent can delegate trust consumption rights to child agents within defined policy boundaries
- Compliance-aware authorization: automatic mapping of consumer context to regulatory framework requirements (GLI-19, EU AI Act, HIPAA)

Layer Mapping: Layer 2 of the Trust Stack

Differentiation from OAuth/RBAC: Traditional authorization controls access to resources. Trust Authorization controls access to trust primitives and produces attestation of correct usage. The attestation is the innovation — not just granting access, but cryptographically recording that access was exercised correctly.

新定位：具有密碼學使用證明的策略閘門存取信任原語。

關鍵權利主張：

- 用於熵消費的策略引擎，根據多維約束（品質下限、來源要求、速率限制、法規背景）評估請求
- 密碼學使用證明：每個消費事件產生不可變的、已簽名的記錄，證明特定消費者在特定策略約束下使用了特定信任原語
- 多代理系統的委託授權：主要代理可以在定義的策略邊界內將信任消費權利委託給子代理
- 合規感知授權：自動將消費者背景對應到法規框架要求（GLI-19、EU AI Act、HIPAA）

層級對應：信任堆疊的第二層

與 OAuth/RBAC 的差異：傳統授權控制對資源的存取。信任授權控制對信任原語的存取，並產生正確使用的證明。證明是創新——不僅僅是授予存取權，而是密碼學地記錄存取權被正確行使。

5.4 — Patent #3: Trust Evidence Layer (VEO)

5.4 — 專利三號：信任證據層（VEO）

Original Filing: Verifiable Entropy Object provisional patent

New Positioning: The standard format for trust evidence — a self-describing, quality-scored, cryptographically signed, optionally blockchain-anchored, independently verifiable digital object.

Key Claims:

- The VEO-1 object format: a structured digital object carrying entropy payload, source provenance array, aggregation metadata, confidence scoring, cryptographic proof, optional blockchain anchor, and lineage references
- Entropy Confidence Score (ECS): a six-dimension weighted scoring system (freshness, diversity, independence, manipulation resistance, verification success, availability) producing a 0–1000 score with defined grade boundaries
- Canonical signing protocol: deterministic transformation of a VEO into a signing payload through deep key sorting and stable serialization, enabling secp256k1 EIP-191 signatures verifiable by any party without network access
- Five-level verification framework: from structural validity through cryptographic verification through blockchain-anchored verification, with distinct policy-failure and invalid states
- Object classification system: VEO-1A (single-source), VEO-1B (composite), VEO-1C (anchored), VEO-1D (decision — reserved)

Layer Mapping: Layer 3 of the Trust Stack

Status: Provisional patent filed. VEO-1 v1.0 specification frozen.

Standards Path: VEO-1 is designed to become an open standard — analogous to JWT for identity claims. The object format, scoring system, signing protocol, and verification logic are fully documented and implementable by any party.

原始申請：可驗證熵物件臨時專利

新定位：信任證據的標準格式——自描述、品質評分、密碼學簽名、可選區塊鏈錨定、可獨立驗證的數位物件。

關鍵權利主張：

- VEO-1 物件格式：攜帶熵載荷、來源出處陣列、聚合元數據、信心評分、密碼學證明、可選區塊鏈錨定和血統參考的結構化數位物件
- 熵信心分數（ECS）：六維度加權評分系統（新鮮度、多樣性、獨立性、抗操縱性、驗證成功率、可用性），產生 0-1000 分數，有定義的等級邊界
- 規範簽名協議：透過深層鍵排序和穩定序列化將 VEO 確定性地轉換為簽名載荷，實現任何方無需網路存取即可驗證的 secp256k1 EIP-191 簽名
- 五級驗證框架：從結構有效性到密碼學驗證到區塊鏈錨定驗證，具有獨立的策略失敗和無效狀態
- 物件分類系統：VEO-1A（單來源）、VEO-1B（複合）、VEO-1C（錨定）、VEO-1D（決策——保留）

層級對應：信任堆疊的第三層

狀態：臨時專利已申請。VEO-1 v1.0 規範已凍結。

標準化路徑：VEO-1 設計為成為開放標準——類似於 JWT 之於身份聲明。物件格式、評分系統、簽名協議和驗證邏輯都已完整記錄，任何方都可實作。

5.5 — Patent #4: Trust Coordination Layer

5.5 — 專利四號：信任協調層

Filing Status: New — to be filed

Positioning: How trust propagates between autonomous agents and across organizational boundaries.

Key Claims:

- Trust chain construction: a protocol for linking child VEOs to parent VEOs via lineage references, creating directed acyclic graphs (DAGs) of trust that enable full decision-tree reconstruction

- Multi-agent trust consensus: a method for combining individual VEOs from multiple autonomous agents into a jointly verifiable consensus VEO, where each participant's contribution is recorded and independently verifiable
- Cross-system trust handoff: a protocol for packaging, transmitting, and verifying trust assertions across organizational trust boundaries without requiring mutual trust between organizations
- Trust propagation invariants: enforcement of non-amplification (child trust $\leq$ parent trust), transparent degradation (reduced sources are recorded, not hidden), and traceability (every VEO in a chain is independently verifiable)

Layer Mapping: Layer 4 of the Trust Stack

Critical Distinction: Trust Coordination is a *protocol*, not an orchestrator. It does not control what agents do. It ensures that trust propagates correctly between them — analogous to TCP for data transmission. This distinction is critical for the patent claims: the invention is the trust propagation protocol, not agent orchestration.

Direction: Horizontal — peer-to-peer trust propagation between autonomous agents and systems.

申請狀態：新——待申請

定位：信任如何在自主代理之間和跨組織邊界傳播。

關鍵權利主張：

- 信任鏈建構：透過血統參考將子 VEO 連結到父 VEO 的協議，建立信任的有向無環圖 (DAG)，實現完整決策樹重建
- 多代理信任共識：將來自多個自主代理的個別 VEO 組合為可聯合驗證的共識 VEO 的方法，每個參與者的貢獻都被記錄且可獨立驗證
- 跨系統信任交接：跨組織信任邊界封裝、傳輸和驗證信任斷言的協議，無需組織間相互信任
- 信任傳播不變量：執行非放大（子信任 $\leq$ 父信任）、透明降級（減少的來源被記錄而非隱藏）和可追溯性（鏈中每個 VEO 都可獨立驗證）

層級對應：信任堆疊的第四層

關鍵區別：信任協調是協議，不是編排器。它不控制代理做什麼。它確保信任在它們之間正確傳播——類似於 TCP 之於資料傳輸。這個區別對專利權利主張至關重要：發明是信任傳播協議，不是代理編排。

方向：水平——自主代理和系統之間的點對點信任傳播。

5.6 — Patent #5: Trust Runtime

5.6 — 專利五號：信任運行時

Filing Status: New — to be filed

Positioning: Governance, compliance, and continuous verification for trust infrastructure.

Key Claims:

- Runtime trust policy enforcement: continuous monitoring and enforcement of system-wide trust policies that cannot be evaluated at the per-request level (chain depth limits, source rotation, anchoring frequency, temporal consistency)
- Compliance attestation generation: structured, machine-readable compliance attestations mapping trust infrastructure operations to regulatory framework requirements (EU AI Act, NIST AI RMF, GLI-19, ISO 42001)

- Trust health monitoring: continuous computation of system-wide trust metrics (average ECS trends, source availability, verification rates, consumption patterns) with anomaly detection and alerting
- Regulatory framework adapters: pluggable modules that map trust lifecycle operations to specific regulatory requirements, enabling automatic compliance reporting for multiple jurisdictions

Layer Mapping: Layer 5 of the Trust Stack

Boundary with Patent #4: The boundary is precise and intentional. **Coordination (Patent #4) is horizontal** — it governs trust flow between peers. **Runtime (Patent #5) is vertical** — it governs trust enforcement from top to bottom. Coordination is a protocol for trust transmission. Runtime is a governance engine for trust assurance. There is no overlap.

Direction: Vertical — policy enforcement and audit from top to bottom of the Trust Stack.

申請狀態：新——待申請

定位：信任基礎設施的治理、合規和持續驗證。

關鍵權利主張：

- 運行時信任策略執行：持續監控和執行無法在逐請求層級評估的系統範圍信任策略（鏈深度限制、來源輪換、錨定頻率、時間一致性）
- 合規證明生成：結構化、機器可讀的合規證明，將信任基礎設施操作對應到法規框架要求（EU AI Act、NIST AI RMF、GLI-19、ISO 42001）
- 信任健康監控：持續計算系統範圍的信任指標（平均 ECS 趨勢、來源可用性、驗證率、消費模式），具有異常偵測和警報
- 法規框架適配器：可插拔模組，將信任生命週期操作對應到特定法規要求，實現多個司法管轄區的自動合規報告

層級對應：信任堆疊的第五層

與專利四號的邊界：邊界是精確且有意的。**協調（專利四號）是水平的**——治理對等方之間的信任流動。**運行時（專利五號）是垂直的**——從上到下治理信任執行。協調是信任傳輸的協議。運行時是信任保證的治理引擎。沒有重疊。

方向：垂直——信任堆疊從上到下的策略執行和稽核。

5.7 — Future Patent Opportunities

5.7 — 未來專利機會

Beyond the core five patents, the Computational Trust architecture creates opportunities for additional filings as the platform matures:

- **Trust Routing:** Intelligent routing of trust requirements to optimal entropy sources based on latency, cost, quality, and regulatory constraints
- **Trust Federation:** Cross-organizational trust interoperability — enabling multiple independent trust providers to form a federated trust network with mutual verification
- **Domain-Specific Trust Objects:** Healthcare Trust Objects (HIPAA-compliant, patient-linked), Financial Trust Objects (SOX-compliant, transaction-linked), Robotics Trust Objects (safety-critical, real-time)
- **Trust Analytics:** Machine learning-based analysis of trust patterns for predictive quality scoring, anomaly detection, and capacity planning

除了核心五件專利之外，計算信任架構隨著平台成熟創造了額外申請的機會：

- **信任路由：**根據延遲、成本、品質和法規約束，將信任需求智慧路由到最佳熵源
- **信任聯邦：**跨組織信任互操作性——使多個獨立信任供應商能夠形成具有相互驗證的聯邦信任網路
- **領域特定信任物件：**醫療信任物件（HIPAA 合規、患者連結）、金融信任物件（SOX 合規、交易連結）、機器人信任物件（安全關鍵、即時）
- **信任分析：**基於機器學習的信任模式分析，用於預測性品質評分、異常偵測和容量規劃

5.8 — Family Coherence

5.8 — 家族一致性

The patent family is designed for coherence, not just coverage:

- **No overlap:** Each patent claims a distinct layer. The boundaries between layers are architecturally defined and technically enforced. An IP challenge to one patent does not weaken the others.
- **Cross-references:** Each patent references the others as related applications, establishing the family relationship and demonstrating the unified architecture.

- **Layered licensing:** The non-overlapping structure enables flexible licensing — individual layer licensing (e.g., a gaming company licensing only Trust Source + Trust Evidence) or full-stack licensing for enterprise deployments.
- **Defensive depth:** Even if one layer's claims are narrowed during prosecution, the remaining four layers maintain independent protection. The architecture is defensible at every level.

專利家族設計追求的是一致性，不僅僅是覆蓋範圍：

- **無重疊：**每件專利主張一個獨特的層。層之間的邊界在架構上定義且技術上強制執行。對一件專利的 IP 挑戰不會削弱其他專利。
- **交叉引用：**每件專利將其他專利引用為相關申請，建立家族關係並展示統一架構。
- **分層授權：**不重疊的結構實現靈活授權——個別層授權（如遊戲公司僅授權信任來源 + 信任證據）或企業部署的全堆疊授權。
- **縱深防禦：**即使一層的權利主張在審查過程中被縮窄，其餘四層維持獨立保護。架構在每個層級都是可防禦的。

Markets

市場

Where Computational Trust Infrastructure creates value

計算信任基礎設施在哪裡創造價值

6.1 — AI Agents (Primary Market)

6.1 — AI 代理（主要市場）

Problem: Autonomous AI agents make non-deterministic decisions — tool selection, response sampling, task routing, resource allocation — with no audit trail for the randomness involved. As agents gain authority over financial, medical, legal, and operational decisions, this becomes a liability for every organization deploying them.

Solution: Every agent decision backed by a VEO. Trust chains across multi-agent systems via the Coordination layer. Compliance attestations via the Runtime layer. SDK integration with major agent frameworks for one-line adoption.

Key Players: OpenAI, Anthropic, Google DeepMind, Microsoft, Cohere, LangChain, CrewAI, AutoGPT, Fixie

Entry Strategy: Open-source SDK + free tier → framework integration PRs → developer adoption → enterprise conversion. The trust lifecycle is invisible to the developer: `openrng.getEntropy({ policy: 'ai-grade' })` — one line, full lifecycle.

問題：自主 AI 代理做出非確定性決策——工具選擇、回應抽樣、任務路由、資源分配——涉及的隨機性沒有稽核軌跡。隨著代理對金融、醫療、法律和營運決策獲得權限，這對每個部署它們的組織都成為一種責任。

解決方案：每個代理決策都由 VEO 支持。透過協調層實現多代理系統間的信任鏈。透過運行時層提供合規證明。與主要代理框架整合的 SDK，實現一行程式碼採用。

關鍵參與者：OpenAI、Anthropic、Google DeepMind、Microsoft、Cohere、LangChain、CrewAI、AutoGPT、Fixie

進入策略：開源 SDK + 免費層 → 框架整合 PR → 開發者採用 → 企業轉換。信任生命週期對開發者是透明的：`openrng.getEntropy({ policy: 'ai-grade' })`——一行程式碼，完整生命週期。

6.2 — Enterprise AI

6.2 — 企業 AI

Problem: Enterprise AI deployments face growing compliance requirements — SOX for financial decisions, GDPR for data processing, EU AI Act for high-risk systems. Current compliance relies on audit logs with circular trust dependencies. Enterprises need infrastructure-level trust assurance, not just documentation.

Solution: Trust Runtime provides automated compliance attestations. Trust Evidence (VEO) creates independently verifiable decision records. The full Trust Stack integrates with enterprise security infrastructure (IAM, SIEM, GRC platforms).

Key Players: AWS, Microsoft Azure, Google Cloud, Databricks, Snowflake, SAP, Oracle, Salesforce

Entry Strategy: Cloud marketplace listings → managed service → enterprise licensing with dedicated support. Target: Chief AI Officers and Chief Compliance Officers.

問題：企業 AI 部署面臨日益增長的合規要求——SOX 用於金融決策、GDPR 用於資料處理、EU AI Act 用於高風險系統。當前合規依賴具有循環信任依賴的稽核日誌。企業需要基礎設施層級的信任保證，而不僅僅是文件。

解決方案：信任運行時提供自動化合規證明。信任證據（VEO）建立可獨立驗證的決策記錄。完整信任堆疊與企業安全基礎設施（IAM、SIEM、GRC 平台）整合。

關鍵參與者：AWS、Microsoft Azure、Google Cloud、Databricks、Snowflake、SAP、Oracle、Salesforce

進入策略：雲端市場上架 → 託管服務 → 具專屬支援的企業授權。目標：首席 AI 長和首席合規長。

6.3 — Gaming & Gambling

6.3 — 遊戲與博弈

Problem: Gaming regulators (GLI, BMM, eCOGRA) require provably fair RNG. Current certification processes test the RNG implementation but rely on the operator's system to log and attest to correct operation — creating the same circular trust dependency found in audit logs.

Solution: VEO provides third-party verifiable proof of fair randomness for every game outcome. ECS scoring provides quantitative quality assurance. Blockchain anchoring provides temporal immutability. Any regulator can verify any outcome independently without trusting the operator.

Key Players: GLI (Gaming Laboratories International), BMM Testlabs, eCOGRA, major igaming operators, casino management system vendors

Entry Strategy: GLI-19 / GLI-33 certification → pilot with progressive operators → industry standard adoption.

問題：遊戲監管機構（GLI、BMM、eCOGRA）要求可證明公平的 RNG。當前認證流程測試 RNG 實作，但依賴操作者的系統來記錄和證明正確操作——造成與稽核日誌相同的循環信任依賴。

解決方案：VEO 為每個遊戲結果提供第三方可驗證的公平隨機性證明。ECS 評分提供量化品質保證。區塊鏈錨定提供時間不可變性。任何監管機構都可以獨立驗證任何結果，無需信任操作者。

關鍵參與者：GLI（遊戲實驗室國際）、BMM Testlabs、eCOGRA、主要 igaming 營運商、賭場管理系統供應商

進入策略：GLI-19 / GLI-33 認證 → 與進步營運商試點 → 行業標準採用。

6.4 — Robotics & Autonomous Systems

6.4 — 機器人與自主系統

Problem: Autonomous robots making physical-world decisions — warehouse allocation, delivery routing, surgical assistance, agricultural optimization — need verifiable decision trails for safety certification, liability, and regulatory compliance.

Solution: Trust Evidence + Trust Coordination for robot swarms. Real-time VEO generation with sub-2ms latency for safety-critical timing. Trust chain construction across multi-robot coordination scenarios.

Entry Strategy: ROS2 plugin → autonomous vehicle/drone frameworks → industrial robotics platforms.

問題：做出實體世界決策的自主機器人——倉庫分配、配送路由、手術輔助、農業優化——需要可驗證的決策軌跡用於安全認證、責任歸屬和法規合規。

解決方案：用於機器人群體的信任證據 + 信任協調。具有低於 2ms 延遲的即時 VEO 生成，滿足安全關鍵時序。跨多機器人協調場景的信任鏈建構。

進入策略：ROS2 插件 → 自主車輛/無人機框架 → 工業機器人平台。

6.5 — Government & Defense

6.5 — 政府與國防

Problem: Government AI systems require the highest-assurance audit trails. Military decision-support systems, intelligence analysis tools, and civilian AI services must comply with frameworks like FedRAMP, NIST 800-53, and DoD-specific requirements.

Solution: Full Trust Stack with compliance adapters for government frameworks. Hardware-backed trust sources (QRNG integration). Air-gapped verification capability (VEO is self-verifying without network access).

Entry Strategy: Government procurement processes → FedRAMP certification → DoD / IC pilot programs.

問題：政府 AI 系統需要最高保證的稽核軌跡。軍事決策支援系統、情報分析工具和民用 AI 服務必須符合 FedRAMP、NIST 800-53 和 DoD 特定要求等框架。

解決方案：具有政府框架合規適配器的完整信任堆疊。硬體支持的信任來源（QRNG 整合）。氣隙隔離驗證能力（VEO 無需網路存取即可自驗證）。

進入策略：政府採購流程 → FedRAMP 認證 → DoD / IC 試點計畫。

6.6 — Finance

6.6 — 金融

Problem: Financial algorithms involving randomness — portfolio rebalancing, Monte Carlo risk models, order routing, A/B testing of trading strategies — face growing regulatory scrutiny. Regulators want proof that algorithmic decisions are fair and unmanipulated.

Solution: VEO for verifiable randomness in financial computations. Trust Runtime for continuous compliance monitoring. Blockchain anchoring for immutable decision records.

Entry Strategy: Protocol integration for DeFi (on-chain VEO verification). API integration for TradFi (enterprise SDK). Regulatory partnership for compliance framework mapping.

問題：涉及隨機性的金融演算法——投資組合再平衡、蒙地卡羅風險模型、訂單路由、交易策略的 A/B 測試——面臨日益增長的監管審查。監管機構希望得到演算法決策公平且未被操縱的證明。

解決方案：用於金融計算中可驗證隨機性的 VEO。用於持續合規監控的信任運行時。用於不可變決策記錄的區塊鏈錨定。

進入策略：DeFi 的協議整合（鏈上 VEO 驗證）。TradFi 的 API 整合（企業 SDK）。合規框架對應的監管合作。

6.7 — Healthcare

6.7 — 醫療保健

Problem: Clinical trial randomization, AI-driven triage, treatment allocation, and diagnostic sampling require auditable entropy with strict regulatory compliance (HIPAA, FDA 21 CFR Part 11, ICH E9 guidelines).

Solution: Healthcare-grade VEOs with HIPAA-compliant anchoring. Trust Runtime with FDA/ICH regulatory adapters. Patient-linkable but privacy-preserving trust chains.

Entry Strategy: Clinical trial platform integration → EHR system partnerships → FDA regulatory pathway.

問題：臨床試驗隨機化、AI 驅動的分診、治療分配和診斷抽樣需要具有嚴格法規合規的可稽核熵（HIPAA、FDA 21 CFR Part 11、ICH E9 指引）。

解決方案：具有 HIPAA 合規錨定的醫療級 VEO。具有 FDA/ICH 法規適配器的信任運行時。可連結患者但保護隱私的信任鏈。

進入策略：臨床試驗平台整合 → EHR 系統夥伴關係 → FDA 法規路徑。

6.8 — Scientific Computing

6.8 — 科學計算

Problem: Monte Carlo simulations, stochastic modeling, and computational experiments require reproducible, auditable entropy for research integrity and peer review.

Solution: VEO provides verifiable entropy with full provenance for research reproducibility. Any reviewer can verify the exact entropy used in a published simulation by checking the VEO independently.

Entry Strategy: Research library integrations (Python/R packages) → journal partnership for reproducibility standards → academic adoption.

問題：蒙地卡羅模擬、隨機建模和計算實驗需要可重現、可稽核的熵以維護研究誠信和同儕審查。

解決方案：VEO 提供具有完整出處的可驗證熵，用於研究可重現性。任何審查者都可以透過獨立檢查 VEO 來驗證已發表模擬中使用的確切熵。

進入策略：研究函式庫整合（Python/R 套件）→ 與期刊合作建立可重現性標準 → 學術採用。

Why This Matters

為什麼這很重要

The case for a new infrastructure category

新基礎設施類別的論證

7.1 — The Historical Pattern

7.1 — 歷史模式

Infrastructure categories are not invented. They are named. They emerge when a problem becomes so pervasive that ad-hoc solutions can no longer contain it, and someone defines the problem precisely enough that a standard solution becomes possible.

TCP/IP did not invent networking. Networks existed. But TCP/IP named the problem — "how do heterogeneous networks interoperate?" — and defined a standard protocol stack that every network could implement. Today, TCP/IP is invisible infrastructure that everything depends on.

TLS did not invent encryption. Encryption existed. But TLS named the problem — "how do two parties establish a secure channel over an untrusted network?" — and defined a standard handshake protocol. Today, HTTPS is so ubiquitous that browsers warn users when it is absent.

OAuth did not invent authorization. Authorization existed. But OAuth named the problem — "how does a user grant an application limited access to their resources without sharing credentials?" — and defined a standard delegation flow. Today, "Sign in with Google" is the default.

JWT did not invent identity claims. Identity claims existed. But JWT named the format — "how do we represent identity as a portable, self-verifying, signed token?" — and defined a standard that every API now speaks.

In each case, the pattern is the same: a pervasive problem exists, individual teams build custom solutions, someone names the problem and defines a standard, and the standard becomes invisible infrastructure. The naming is the breakthrough. Before the name exists, the problem is fragmented. After the name exists, the category crystallizes.

基礎設施類別不是被發明的。它們是被命名的。它們在一個問題變得如此普遍以至於臨時解決方案無法再應對時出現，而某人足夠精確地定義了問題，使標準解決方案成為可能。

TCP/IP 沒有發明網路。網路已經存在。但 TCP/IP 命名了問題——「異構網路如何互操作？」——並定義了每個網路都可以實作的標準協議堆疊。今天，TCP/IP 是一切都依賴的隱形基礎設施。

TLS 沒有發明加密。加密已經存在。但 TLS 命名了問題——「兩方如何在不受信任的網路上建立安全通道？」——並定義了標準握手協議。今天，HTTPS 如此普遍，以至於瀏覽器在缺少它時會警告用戶。

OAuth 沒有發明授權。授權已經存在。但 OAuth 命名了問題——「用戶如何在分享憑證的情況下授予應用程式有限的資源存取權？」——並定義了標準委託流程。今天，「使用 Google 登入」是預設選項。

JWT 沒有發明身份聲明。身份聲明已經存在。但 JWT 命名了格式——「如何將身份表示為可攜帶、自驗證、已簽名的令牌？」——並定義了每個 API 現在都使用的標準。

在每種情況下，模式是相同的：普遍的問題存在、個別團隊建構自定解決方案、某人命名問題並定義標準、標準成為隱形基礎設施。命名就是突破。在名稱存在之前，問題是碎片化的。在名稱存在之後，類別結晶。

7.2 — Computational Trust Is the Next Category

7.2 — 計算信任是下一個類別

The problem is here. Autonomous systems are proliferating faster than the trust infrastructure to support them. Every major organization deploying AI agents is building custom solutions for decision auditability — custom logging, custom attestation, custom compliance mapping. These solutions are fragmented, incompatible, and insufficient.

The regulatory demand is here. Every major regulatory body — EU, US, UK, Singapore, Japan — is independently converging on the same requirement: AI decisions must be auditable and verifiable. But no regulatory framework specifies how, because the infrastructure to do so does not exist.

The name is here. **Computational Trust** — the ability to independently verify that a computational process occurred correctly, fairly, and without manipulation. This is not security (protecting the perimeter), not privacy (protecting data), not compliance (following rules). It is a distinct infrastructure layer that sits alongside all three.

OpenRNG is naming this category and building its first implementation. The Trust Stack defines the architecture. The Trust Lifecycle defines the operations. The VEO-1 specification defines the object format. The patent family protects the intellectual property. The SDK and open standard enable adoption.

問題已經在這裡。自主系統的擴散速度超過了支撐它們的信任基礎設施。每個部署 AI 代理的主要組織都在為決策可稽核性建構自定解決方案——自定日誌、自定證明、自定合規對應。這些解決方案是碎片化的、不相容的、不充分的。

監管需求已經在這裡。每個主要監管機構——歐盟、美國、英國、新加坡、日本——都獨立地趨同於相同的要求：AI 決策必須是可稽核和可驗證的。但沒有任何監管框架指定如何做到，因為做到這一點的基礎設施不存在。

名稱已經在這裡。**計算信任**——獨立驗證計算過程正確、公平且未被操縱地發生的能力。這不是安全（保護邊界）、不是隱私（保護資料）、不是合規（遵循規則）。它是一個獨特的基礎設施層，與這三者並列存在。

OpenRNG 正在命名這個類別並建構其第一個實作。信任堆疊定義架構。信任生命週期定義操作。VEO-1 規範定義物件格式。專利家族保護智慧財產權。SDK 和開放標準實現採用。

7.3 — The Infrastructure Test

7.3 — 基礎設施測試

Four questions determine whether something is a product or infrastructure:

四個問題決定某物是產品還是基礎設施：

Question 問題	Answer 回答
Does it solve a problem that will only get bigger? 它解決的問題是否只會越來越大？	Yes. Autonomous systems are accelerating. Every new agent deployed increases the trust gap. 是。自主系統正在加速。每個新部署的代理都增加了信任缺口。
Does it require a new primitive, not a better version of an existing one? 它是否需要新的原語，而非現有原語的更好版本？	Yes. Trust objects (VEO) do not exist. This is not a better RNG — it is a new category of object. 是。信任物件（VEO）不存在。這不是更好的 RNG ——這是一個新類別的物件。
Can it become invisible — infrastructure that everything depends on but nobody sees? 它能否變得隱形——一切都依賴但沒人看見的基礎設施？	Yes. Like TLS today: developers call an API and trust happens. The lifecycle is invisible. 是。像今天的 TLS：開發者呼叫 API，信任就發生了。生命週期是隱形的。
Does it have a natural standards path? 它是否有自然的標準化路徑？	Yes. VEO → open specification → IETF/W3C submission. The path mirrors JWT, OAuth, and TLS. 是。VEO → 開放規範 → IETF/W3C 提交。路徑與 JWT、OAuth 和 TLS 相似。

Computational Trust passes all four tests. It is infrastructure, not a product.

計算信任通過所有四項測試。它是基礎設施，不是產品。

7.4 — The Five-Year Vision

7.4 — 五年願景

In five years, the OpenRNG ecosystem should look like this:

五年後，OpenRNG 生態系統應該是這樣的：

OpenRNG – Computational Trust Infrastructure



Each layer derives from the one above it. The Bible defines the world. The Whitepaper explains the science. The OTS specifies the protocol. The Reference Implementation proves it works. The SDK makes it adoptable. The Open Standard makes it universal. The Patent Family protects it. Enterprise Products monetize it.

This is not a product roadmap. It is a category creation sequence — the same sequence followed by TLS, OAuth, Docker, and Kubernetes. The order matters: you cannot standardize what you have not specified, you cannot specify what you have not explained, and you cannot explain what you have not defined.

每一層都源自其上方的一層。聖經定義世界。白皮書解釋科學。OTS 規定協議。參考實作證明它有效。SDK 使其可採用。開放標準使其普遍。專利家族保護它。企業產品將其貨幣化。

這不是產品路線圖。這是類別創建序列——TLS、OAuth、Docker 和 Kubernetes 遵循的相同序列。順序很重要：你無法標準化你未規定的東西，你無法規定你未解釋的東西，你無法解釋你未定義的東西。

7.5 — What Success Looks Like

7.5 — 成功的樣貌

- **"VEO"** becomes a term like "JWT" — every developer building AI agent systems knows what it is and expects to use it.
- **Major cloud providers** offer VEO verification as a managed service — "AWS Trust Verify," "Azure Computational Trust" — the same way they offer KMS, HSM, and certificate management today.
- **Regulatory frameworks** reference Computational Trust as a compliance mechanism — "AI decisions shall be accompanied by Verifiable Trust Evidence" appears in regulatory guidance, the way "data shall be encrypted in transit" appears today.
- **Agent frameworks** include trust lifecycle integration as a core feature — LangChain, CrewAI, and future frameworks ship with built-in VEO support, the way web frameworks ship with HTTPS support.
- **OpenRNG** is to computational trust what Cloudflare is to the edge, what Stripe is to payments, what Let's Encrypt is to TLS: the default infrastructure that makes the category accessible.

This is not a prediction. It is a design target. Every decision — architectural, legal, commercial — is made with this end state in mind. The Bible you are reading defines the forest. The patents, specifications, SDKs, and products are the trees that grow within it.

- 「**VEO**」成為像「JWT」一樣的術語——每個建構 AI 代理系統的開發者都知道它是什麼，並期望使用它。
- 主要雲端供應商提供 VEO 驗證作為託管服務——「AWS Trust Verify」、「Azure Computational Trust」——與今天他們提供 KMS、HSM 和憑證管理的方式相同。
- 監管框架將計算信任引用為合規機制——「AI 決策應附帶可驗證信任證據」出現在監管指引中，如同今天「資料在傳輸中應加密」的出現方式。
- 代理框架將信任生命週期整合為核心功能——LangChain、CrewAI 和未來的框架出廠即內建 VEO 支援，如同 Web 框架出廠即支援 HTTPS。
- **OpenRNG** 之於計算信任，如同 Cloudflare 之於邊緣、Stripe 之於支付、Let's Encrypt 之於 TLS：使類別可及的預設基礎設施。

這不是預測。這是設計目標。每個決策——架構的、法律的、商業的——都以此最終狀態為依歸。你正在閱讀的聖經定義了森林。專利、規範、SDK 和產品是在其中生長的樹木。



OpenRNG — Computational Trust Infrastructure v1.0

Bilingual Edition — English / 繁體中文

June 2026

CONFIDENTIAL — FOR IP COUNSEL REVIEW

© 2026 OpenRNG. All rights reserved.